

NUMPYTASK1.PY

```
import numpy as np

print("=== Task 1: NumPy Introduction & Arrays ===")

array_1d = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("\n--- 1D Array Details ---")
print("Array Elements:", array_1d)
print("Shape of Array:", array_1d.shape)
print("Size of Array :", array_1d.size)
print("Data Type      :", array_1d.dtype)

array_2d = np.array([[1, 2, 3], [4, 5, 6]])

print("\n--- 2D Array Details ---")
print("2D Array:\n", array_2d)
print("Shape of 2D Array:", array_2d.shape)
```

OUTPUTTASK1

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\numpytask1.py

=== Task 1: NumPy Introduction & Arrays ===

--- 1D Array Details ---

Array Elements: [10 20 30 40 50 60 70 80 90 100]

Shape of Array: (10,)

Size of Array : 10

Data Type : int64

--- 2D Array Details ---

2D Array:

[[1 2 3]

[4 5 6]]

Shape of 2D Array: (2, 3)

Process finished with exit code 0

Numpytask2.py

```
import numpy as np
print("=== Task 2: NumPy Indexing, Slicing & Reshaping ===")
arr_1d = np.array([10, 20, 30, 40, 50, 60])
print("\n--- 1D Indexing & Slicing ---")
print("Original 1D Array:", arr_1d)
print("Element at index 2:", arr_1d[2])
print("Sliced portion [1:4]:", arr_1d[1:4])
arr_2d = np.array([[11, 12, 13],
                   [31, 32, 33]])
print("\n--- 2D Array Access ---")
print("Original 2D Array:\n", arr_2d)
print("Access Specific Row (Row 1):", arr_2d[1])
print("Access Specific Column (Col 2):", arr_2d[:, 2])
reshaped_arr = arr_1d.reshape(2, 3)
print("\n--- Reshaping Array ---")
print("Original 1D Array :", arr_1d)
print("Reshaped to 2D (2x3):\n", reshaped_arr)
```

output2

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\numpytask2.py

=== Task 2: NumPy Indexing, Slicing & Reshaping ===

--- 1D Indexing & Slicing ---

Original 1D Array: [10 20 30 40 50 60]

Element at index 2: 30

Sliced portion [1:4]: [20 30 40]

--- 2D Array Access ---

Original 2D Array:

[[11 12 13]

[31 32 33]]

Access Specific Row (Row 1): [31 32 33]

Access Specific Column (Col 2): [13 33]

--- Reshaping Array ---

Original 1D Array : [10 20 30 40 50 60]

Reshaped to 2D (2x3):

[[10 20 30]

[40 50 60]]

Process finished with exit code 0

Numpytask3.py

```
import numpy as np

print("=== Task 3: NumPy Mathematical & Statistical Operations ===")

arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([2, 4, 5, 10])

print("\n--- 1. Mathematical Operations ---")
print("Array 1:", arr1)
print("Array 2:", arr2)

print("Addition (+):      ", arr1 + arr2)
print("Subtraction (-):   ", arr1 - arr2)
print("Multiplication (*):", arr1 * arr2)
print("Division (/):       ", arr1 / arr2)

dataset = np.array([5, 12, 15, 20, 25, 30, 45])

print("\n--- 2. Statistical Functions ---")
print("Dataset:", dataset)
print("Mean      :", np.mean(dataset))
print("Median    :", np.median(dataset))
print("Minimum   :", np.min(dataset))
print("Maximum   :", np.max(dataset))
print("Standard Deviation :", np.round(np.std(dataset), 2))
print("Sum       :", np.sum(dataset))
```

output3

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\numpytask3.py

=== Task 3: NumPy Mathematical & Statistical Operations ===

--- 1. Mathematical Operations ---

Array 1: [10 20 30 40]

Array 2: [2 4 5 10]

Addition (+): [12 24 35 50]

Subtraction (-): [8 16 25 30]

Multiplication (*): [20 80 150 400]

Division (/): [5. 5. 6. 4.]

--- 2. Statistical Functions ---

Dataset: [5 12 15 20 25 30 45]

Mean : 21.714285714285715

Median : 20.0

Minimum : 5

Maximum : 45

Standard Deviation : 12.21

Sum : 152

Process finished with exit code 0

Nmupytask4.py

```

import pandas as pd

print("=== Task 4: Pandas Series & DataFrame ===")
print("\n--- 1. Pandas Series ---")
data_list = [10, 20, 30, 40, 50]
my_series = pd.Series(data_list)
print(my_series)
print("\n--- 2. Creating DataFrame (Student Info) ---")
student_data = {
    "Name": ["Ayush", "Rahul", "Amit"],
    "Roll_No": ["0884", "0885", "0886"],
    "Branch": ["CS", "IT", "CS"]
}
df = pd.DataFrame(student_data)
print("DataFrame:\n", df)
print("\nColumn Names:", df.columns.tolist())
print("Index Details:", df.index)
print("\n--- 3. Adding New Column & Updating ---")
df["City"] = ["Bhopal", "Indore", "Gwalior"]
print("Updated DataFrame:\n", df)

```

output4

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\pandastask4.py

=== Task 4: Pandas Series & DataFrame ===

--- 1. Pandas Series ---

0 10

1 20

2 30

3 40

4 50

dtype: int64

--- 2. Creating DataFrame (Student Info) ---

DataFrame:

	Name	Roll_No	Branch
0	Ayush	0884	CS
1	Rahul	0885	IT
2	Amit	0886	CS

Column Names: ['Name', 'Roll_No', 'Branch']

Index Details: RangeIndex(start=0, stop=3, step=1)

--- 3. Adding New Column & Updating ---

Updated DataFrame:

	Name	Roll_No	Branch	City
0	Ayush	0884	CS	Bhopal
1	Rahul	0885	IT	Indore
2	Amit	0886	CS	Gwalior

Process finished with exit code 0

Pandastask5.py

```
import pandas as pd
print("=== Task 5: Reading & Inspecting Data ===")
csv_data = """Roll_No,Name,Branch,Marks,City
101,Ayush,CS,85,Bhopal
102,Rahul,IT,78,Indore
103,Amit,CS,92,Gwalior
104,Priya,EC,64,Jabalpur
105,Sneha,CS,88,Ujjain
106,Rohit,IT,71,Bhopal
107,Vikas,EC,55,Indore
"""
with open("students_dataset.csv", "w") as f:
    f.write(csv_data.strip())
```

```

df = pd.read_csv("students_dataset.csv")
print("\n--- 1. First 5 rows using head() ---")
print(df.head())
print("\n--- 2. Last 5 rows using tail() ---")
print(df.tail())
print("\n--- 3. Shape of Dataset (Rows & Columns) ---")
print(f"Number of rows and columns: {df.shape}")
print("\n--- 4. Column Names ---")
print(df.columns.tolist())
print("\n--- 5. Data Types using dtypes ---")
print(df.dtypes)
print("\n--- 6. Info using info() ---")
df.info()
print("\n--- 7. Description using describe() ---")
print(df.describe())

```

output5

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\pandastask5.py

=== Task 5: Reading & Inspecting Data ===

--- 1. First 5 rows using head() ---

	Roll_No	Name	Branch	Marks	City
0	101	Ayush	CS	85	Bhopal
1	102	Rahul	IT	78	Indore
2	103	Amit	CS	92	Gwalior
3	104	Priya	EC	64	Jabalpur
4	105	Sneha	CS	88	Ujjain

--- 2. Last 5 rows using tail() ---

	Roll_No	Name	Branch	Marks	City
2	103	Amit	CS	92	Gwalior
3	104	Priya	EC	64	Jabalpur

```
4    105 Sneha   CS   88   Ujjain
5    106 Rohit   IT   71   Bhopal
6    107 Vikas   EC   55   Indore
```

--- 3. Shape of Dataset (Rows & Columns) ---

Number of rows and columns: (7, 5)

--- 4. Column Names ---

```
['Roll_No', 'Name', 'Branch', 'Marks', 'City']
```

--- 5. Data Types using dtypes ---

```
Roll_No    int64
```

```
Name       str
```

```
Branch     str
```

```
Marks      int64
```

```
City       str
```

```
dtype: object
```

--- 6. Info using info() ---

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 7 entries, 0 to 6
```

```
Data columns (total 5 columns):
```

```
#   Column  Non-Null Count  Dtype
```

```
---  -----  -
```

```
0  Roll_No  7 non-null  int64
1  Name     7 non-null  str
2  Branch   7 non-null  str
3  Marks    7 non-null  int64
4  City     7 non-null  str
```

dtypes: int64(2), str(3)

memory usage: 412.0 bytes

--- 7. Description using describe() ---

```
      Roll_No  Marks
count  7.000000  7.000000
mean   104.000000  76.142857
std     2.160247  13.508375
min    101.000000  55.000000
25%    102.500000  67.500000
50%    104.000000  78.000000
75%    105.500000  86.500000
max    107.000000  92.000000
```

Process finished with exit code 0

Pandastask6.py

```
import pandas as pd
print("=== Task 6: Selecting, Filtering & Sorting Data ===")
student_data = {
    "Roll_No": [101, 102, 103, 104, 105, ],
    "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
    "Branch": ["CS", "IT", "CS", "EC", "CS"],
```

```

    "Marks": [85, 78, 92, 64, 88]
}
df = pd.DataFrame(student_data)
print("Original DataFrame:\n", df)
print("\n--- 1. Selecting Specific Columns ---")
print(df[["Name", "Marks"]])
print("\n--- 2. Selecting Specific Rows (Using iloc) ---")
print(df.iloc[0:2])
print("\n--- 3. Single Filter (Marks > 80) ---")
single_filter = df[df["Marks"] > 80]
print(single_filter)
print("\n--- 4. Multiple Filters (Branch == CS AND Marks > 85) ---")
multiple_filter = df[(df["Branch"] == "CS") & (df["Marks"] > 85)]
print(multiple_filter)
print("\n--- 5. Sorting in Ascending Order (By Marks) ---")
sort_asc = df.sort_values(by="Marks", ascending=True)
print(sort_asc)
print("\n--- 6. Sorting in Descending Order (By Marks) ---")
sort_desc = df.sort_values(by="Marks", ascending=False)
print(sort_desc)

```

output6

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\pandastask6.py

=== Task 6: Selecting, Filtering & Sorting Data ===

Original DataFrame:

	Roll_No	Name	Branch	Marks
0	101	Ayush	CS	85
1	102	Rahul	IT	78
2	103	Amit	CS	92
3	104	Priya	EC	64
4	105	Sneha	CS	88

--- 1. Selecting Specific Columns ---

	Name	Marks
0	Ayush	85

1	Rahul	78
2	Amit	92
3	Priya	64
4	Sneha	88

--- 2. Selecting Specific Rows (Using iloc) ---

	Roll_No	Name	Branch	Marks
0	101	Ayush	CS	85
1	102	Rahul	IT	78

--- 3. Single Filter (Marks > 80) ---

	Roll_No	Name	Branch	Marks
0	101	Ayush	CS	85
2	103	Amit	CS	92
4	105	Sneha	CS	88

--- 4. Multiple Filters (Branch == CS AND Marks > 85) ---

	Roll_No	Name	Branch	Marks
2	103	Amit	CS	92
4	105	Sneha	CS	88

--- 5. Sorting in Ascending Order (By Marks) ---

	Roll_No	Name	Branch	Marks
3	104	Priya	EC	64

1	102	Rahul	IT	78
0	101	Ayush	CS	85
4	105	Sneha	CS	88
2	103	Amit	CS	92

--- 6. Sorting in Descending Order (By Marks) ---

	Roll_No	Name	Branch	Marks
2	103	Amit	CS	92
4	105	Sneha	CS	88
0	101	Ayush	CS	85
1	102	Rahul	IT	78
3	104	Priya	EC	64

Process finished with exit code 0

Pandastask7.py

```
import pandas as pd
import numpy as np

print("=== Task 7: Handling Missing Values ===")
student_data = {
    "Roll_No": [101, 102, 103, 104, 105],
    "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
    "Marks": [85, np.nan, 92, np.nan, 76],
    "City": ["Bhopal", "Indore", None, "Jabalpur", "Bhopal"]
}
df = pd.DataFrame(student_data)
print("\n--- 1. Original Dataset Before Handling ---")
print(df)
print("\n--- 2. Identifying Missing Values (isnull) ---")
print(df.isnull())
print("\n--- 3. Count of Missing Values in Each Column ---")
print(df.isnull().sum())
print("\n--- 4. Dataset After Filling Missing Values (fillna) ---")
df_filled = df.copy()
df_filled["Marks"] = df_filled["Marks"].fillna(df["Marks"].mean())
df_filled["City"] = df_filled["City"].fillna("Unknown")
```

```
print(df_filled)
print("\n--- 5. Dataset After Removing Rows with Missing Values (dropna) ---")
df_dropped = df.dropna()
print(df_dropped)
```

output7

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\pandastask7.py

=== Task 7: Handling Missing Values ===

--- 1. Original Dataset Before Handling ---

	Roll_No	Name	Marks	City
0	101	Ayush	85.0	Bhopal
1	102	Rahul	NaN	Indore
2	103	Amit	92.0	NaN
3	104	Priya	NaN	Jabalpur
4	105	Sneha	76.0	Bhopal

--- 2. Identifying Missing Values (isnull) ---

	Roll_No	Name	Marks	City
0	False	False	False	False
1	False	False	True	False
2	False	False	False	True
3	False	False	True	False
4	False	False	False	False

--- 3. Count of Missing Values in Each Column ---

Roll_No 0

Name 0

Marks 2

City 1

dtype: int64

--- 4. Dataset After Filling Missing Values (fillna) ---

	Roll_No	Name	Marks	City
0	101	Ayush	85.000000	Bhopal
1	102	Rahul	84.333333	Indore
2	103	Amit	92.000000	Unknown
3	104	Priya	84.333333	Jabalpur
4	105	Sneha	76.000000	Bhopal

--- 5. Dataset After Removing Rows with Missing Values (dropna) ---

	Roll_No	Name	Marks	City
0	101	Ayush	85.0	Bhopal
4	105	Sneha	76.0	Bhopal

Process finished with exit code 0

Pandas8task.py

```

import pandas as pd
import numpy as np

print("=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===")
print("\n--- 1. Merging two DataFrames ---")

df_names = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Name": ["Ayush", "Rahul", "Amit"]
})
df_marks = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Marks": [85, 78, 92]
})
merged_df = pd.merge(df_names, df_marks, on="Roll_No")
print(merged_df)
print("\n--- 2. Concatenating two DataFrames ---")
batch1 = pd.DataFrame({"Name": ["Ayush", "Rahul"], "Branch": ["CS", "IT"]})
batch2 = pd.DataFrame({"Name": ["Amit", "Priya"], "Branch": ["CS", "EC"]})
concated_df = pd.concat([batch1, batch2], ignore_index=True)
print(concated_df)
print("\n--- 3. GroupBy Operations ---")
sales_data = pd.DataFrame({
    "Branch": ["CS", "IT", "CS", "IT", "CS"],
    "Students": ["ayush", "rahul", "amit", "priya", "sneha"],
    "Fees": [5000, 6000, 5200, 5800, 5100]
})
grouped_sum = sales_data.groupby("Branch")["Students"].sum()
grouped_mean = sales_data.groupby("Branch")["Fees"].mean()
print("Total Students by Branch:\n", grouped_sum)
print("\nAverage Fees by Branch:\n", grouped_mean)
print("\n--- 4. Creating Pivot Table ---")
store_data = pd.DataFrame({
    "Region": ["North", "North", "South", "South", "North"],
    "Item": ["Laptop", "Mobile", "Laptop", "Mobile", "Laptop"],
    "Sales": [15000, 8000, 12000, 9000, 16000]
})
pivot_table = pd.pivot_table(store_data, values="Sales", index="Region",
columns="Item", aggfunc=np.sum)
print(pivot_table)

```

output8

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\pandas8task.py

=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===

--- 1. Merging two DataFrames ---

Roll_No	Name	Marks
---------	------	-------

```
0    101 Ayush    85
1    102 Rahul    78
2    103 Amit     92
```

--- 2. Concatenating two DataFrames ---

Name Branch

```
0 Ayush    CS
1 Rahul    IT
2 Amit     CS
3 Priya    EC
```

--- 3. GroupBy Operations ---

Total Students by Branch:

Branch

```
CS    ayushamitsneha
```

```
IT     rahulpriya
```

Name: Students, dtype: str

Average Fees by Branch:

Branch

```
CS    5100.0
```

```
IT    5900.0
```

Name: Fees, dtype: float64

--- 4. Creating Pivot Table ---

Item Laptop Mobile

Region

North 31000 8000

South 12000 9000

Process finished with exit code 0

Pandas9task.py

```
import pandas as pd
import numpy as np

print("=== Task 9: Exporting Data ===")

cleaned_data = {
    "Roll_No": [101, 102, 103, 104, 105],
    "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
    "Marks": [85.0, 84.33, 92.0, 84.33, 76.0], # साफ़ किया हुआ डेटा
    "City": ["Bhopal", "Indore", "Unknown", "Jabalpur", "Bhopal"]
}

df_final = pd.DataFrame(cleaned_data)
df_final.to_csv("final_processed_data.csv", index=False)
print("\n✓ Final DataFrame successfully exported to
'final_processed_data.csv'")

df_verify = pd.read_csv("final_processed_data.csv")
print("\n--- Verifying Exported File Contains the Processed Data ---")
print(df_verify)
```

output9

C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\9task.py

=== Task 9: Exporting Data ===

✓ Final DataFrame successfully exported to 'final_processed_data.csv'

--- Verifying Exported File Contains the Processed Data ---

	Roll_No	Name	Marks	City
0	101	Ayush	85.00	Bhopal
1	102	Rahul	84.33	Indore
2	103	Amit	92.00	Unknown
3	104	Priya	84.33	Jabalpur
4	105	Sneha	76.00	Bhopal

Process finished with exit code 0

Pandas8and9task.py

```
import pandas as pd
import numpy as np

print("=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===")
print("\n--- 1. Merging two DataFrames ---")

df_names = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Name": ["Ayush", "Rahul", "Amit"]
})
df_marks = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Marks": [85, 78, 92]
})
merged_df = pd.merge(df_names, df_marks, on="Roll_No")
print(merged_df)
print("\n--- 2. Concatenating two DataFrames ---")
batch1 = pd.DataFrame({"Name": ["Ayush", "Rahul"], "Branch": ["CS", "IT"]})
batch2 = pd.DataFrame({"Name": ["Amit", "Priya"], "Branch": ["CS", "EC"]})
concatated_df = pd.concat([batch1, batch2], ignore_index=True)
print(concatated_df)
print("\n--- 3. GroupBy Operations ---")
sales_data = pd.DataFrame({
    "Branch": ["CS", "IT", "CS", "IT", "CS"],
    "Students": ["ayush", "rahul", "amit", "priya", "sneha"],
    "Fees": [5000, 6000, 5200, 5800, 5100]
})
grouped_sum = sales_data.groupby("Branch")["Students"].sum()
grouped_mean = sales_data.groupby("Branch")["Fees"].mean()
print("\nTotal Students by Branch:\n", grouped_sum)
print("\nAverage Fees by Branch:\n", grouped_mean)
print("\n--- 4. Creating Pivot Table ---")
store_data = pd.DataFrame({
```

```

    "Region": ["North", "North", "South", "South", "North"],
    "Item": ["Laptop", "Mobile", "Laptop", "Mobile", "Laptop"],
    "Sales": [15000, 8000, 12000, 9000, 16000]
})
pivot_table = pd.pivot_table(store_data, values="Sales", index="Region",
columns="Item", aggfunc=np.sum)
print(pivot_table)
print("\n=== Task 9: Exporting Data ===")
pivot_table.to_csv("final_processed_data.csv", index=False)
print("✓ Final DataFrame successfully exported to
'final_processed_data.csv'")
df_verify = pd.read_csv("final_processed_data.csv")
print("\n--- Verifying Exported File Contains the Processed Data ---")
print(df_verify)

```

output8and9

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\pandas9task.py

=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===

--- 1. Merging two DataFrames ---

	Roll_No	Name	Marks
0	101	Ayush	85
1	102	Rahul	78
2	103	Amit	92

--- 2. Concatenating two DataFrames ---

	Name	Branch
0	Ayush	CS
1	Rahul	IT
2	Amit	CS
3	Priya	EC

--- 3. GroupBy Operations ---

Total Students by Branch:

Branch

CS ayushamitsneha

IT rahulpriya

Name: Students, dtype: str

Average Fees by Branch:

Branch

CS 5100.0

IT 5900.0

Name: Fees, dtype: float64

--- 4. Creating Pivot Table ---

Item Laptop Mobile

Region

North 31000 8000

South 12000 9000

=== Task 9: Exporting Data ===

✓ Final DataFrame successfully exported to 'final_processed_data.csv'

--- Verifying Exported File Contains the Processed Data ---

Laptop Mobile

0 31000 8000

1 12000 9000

Process finished with exit code 0

Pandastask10.py

```
import pandas as pd
import numpy as np

print("=== Task 10: Mini Data Analysis Project (Employee Dataset) ===")
raw_data = {
    "Emp_ID": [101, 102, 103, 104, 105, 106],
    "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha", "Vikas"],
    "Department": ["IT", "HR", "IT", "HR", "IT", "HR"],
    "Salary": [60000, np.nan, 75000, 45000, np.nan, 50000],
    "City": ["Bhopal", "Indore", None, "Jabalpur", "Bhopal", "Indore"]
}
df = pd.DataFrame(raw_data)
print("\n--- 1. Loaded Dataset ---")
print(df)
print("\n--- 2. Data Inspection (First 3 Rows) ---")
print(df.head(3))
print("\nDataset Shape (Rows, Columns):", df.shape)
print("\n--- 3. Identifying Missing Values Per Column ---")
print(df.isnull().sum())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
df["City"] = df["City"].fillna("Unknown")
print("\nDataset After Handling Missing Values:")
print(df)
print("\n--- 4. Filtering Data (Employees with Salary > 52000) ---")
high_earners = df[df["Salary"] > 52000]
print(high_earners[["Name", "Department", "Salary"]])
print("\n--- 5. Sorting Dataset by Salary (High to Low) ---")
sorted_df = df.sort_values(by="Salary", ascending=False)
print(sorted_df)
print("\n--- 6. GroupBy Department (Average Salary) ---")
avg_salary_dept = df.groupby("Department")["Salary"].mean()
print(avg_salary_dept)
print("\n--- 7. Creating Pivot Table (Total Salary by City & Department) ---")
pivot = pd.pivot_table(df, values="Salary", index="City",
                        columns="Department", aggfunc=np.sum, fill_value=0)
print(pivot)
print("\n--- 8. Useful Insights ---")
print(f"📊 Highest Paid Employee Salary: {df['Salary'].max()}")
print(f"💰 Total Monthly Payroll of Company: {df['Salary'].sum()}")
df.to_csv("final_project_employee_data.csv", index=False)
```

```
print("\n✓ 9. Cleaned dataset successfully exported to  
'final_project_employee_data.csv'")
```

output10

C:\Users\hp\Desktop\.venv\Scripts\python.exe

C:\Users\hp\Desktop\task10.py

=== Task 10: Mini Data Analysis Project (Employee Dataset) ===

--- 1. Loaded Dataset ---

	Emp_ID	Name	Department	Salary	City
0	101	Ayush	IT	60000.0	Bhopal
1	102	Rahul	HR	NaN	Indore
2	103	Amit	IT	75000.0	NaN
3	104	Priya	HR	45000.0	Jabalpur
4	105	Sneha	IT	NaN	Bhopal
5	106	Vikas	HR	50000.0	Indore

--- 2. Data Inspection (First 3 Rows) ---

	Emp_ID	Name	Department	Salary	City
0	101	Ayush	IT	60000.0	Bhopal
1	102	Rahul	HR	NaN	Indore
2	103	Amit	IT	75000.0	NaN

Dataset Shape (Rows, Columns): (6, 5)

--- 3. Identifying Missing Values Per Column ---

Emp_ID 0

Name 0

Department 0

Salary 2

City 1

dtype: int64

Dataset After Handling Missing Values:

	Emp_ID	Name	Department	Salary	City
0	101	Ayush	IT	60000.0	Bhopal
1	102	Rahul	HR	57500.0	Indore
2	103	Amit	IT	75000.0	Unknown
3	104	Priya	HR	45000.0	Jabalpur
4	105	Sneha	IT	57500.0	Bhopal
5	106	Vikas	HR	50000.0	Indore

--- 4. Filtering Data (Employees with Salary > 52000) ---

	Name	Department	Salary
0	Ayush	IT	60000.0
1	Rahul	HR	57500.0
2	Amit	IT	75000.0
4	Sneha	IT	57500.0

--- 5. Sorting Dataset by Salary (High to Low) ---

	Emp_ID	Name	Department	Salary	City
2	103	Amit	IT	75000.0	Unknown
0	101	Ayush	IT	60000.0	Bhopal
1	102	Rahul	HR	57500.0	Indore
4	105	Sneha	IT	57500.0	Bhopal
5	106	Vikas	HR	50000.0	Indore
3	104	Priya	HR	45000.0	Jabalpur

--- 6. GroupBy Department (Average Salary) ---

Department

HR 50833.333333

IT 64166.666667

Name: Salary, dtype: float64

--- 7. Creating Pivot Table (Total Salary by City & Department) ---

Department	HR	IT
City		
Bhopal	0.0	117500.0
Indore	107500.0	0.0
Jabalpur	45000.0	0.0
Unknown	0.0	75000.0

--- 8. Useful Insights ---

📊 Highest Paid Employee Salary: 75000.0

💰 Total Monthly Payroll of Company: 345000.0

✓ 9. Cleaned dataset successfully exported to
'final_project_employee_data.csv'

Process finished with exit code 0

```
Desktop
Run numpytask1 x
numpytask1.py x
...
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\numpytask1.py
=== Task 1: NumPy Introduction & Arrays ===

--- 1D Array Details ---
Array Elements: [ 10  20  30  40  50  60  70  80  90 100]
Shape of Array: (10,)
Size of Array : 10
Data Type      : int64

--- 2D Array Details ---
2D Array:
[[1 2 3]
 [4 5 6]]
Shape of 2D Array: (2, 3)

Process finished with exit code 0
|

Desktop > numpytask1.py
10:42 CRLF U
3 cm o
```

```
Version Control v
numpytask1.py x
numpytask1 v
p\Desktop
mp.lnk
1
2 import numpy as np
3
4 print("=== Task 1: NumPy Introduction & Arrays ===")
5
6 array_1d = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
7
8 print("\n--- 1D Array Details ---")
9 print("Array Elements:", array_1d)
10 print("Shape of Array:", array_1d.shape)
11 print("Size of Array :", array_1d.size)
12 print("Data Type      :", array_1d.dtype)
13
14 array_2d = np.array([[1, 2, 3], [4, 5, 6]])
15
16 print("\n--- 2D Array Details ---")
17 print("2D Array:\n", array_2d)
18 print("Shape of 2D Array:", array_2d.shape)
```

task2.py ▾ Version Control ▾

numpytask2 ▾ ▶ ⚙️ ⋮ @ 🔍 ⚙️ Trial

numpytask1.py

numpytask2.py ×

```
1 import numpy as np
2 print("=== Task 2: NumPy Indexing, Slicing & Reshaping ===")
3 arr_1d = np.array([10, 20, 30, 40, 50, 60])
4 print("\n--- 1D Indexing & Slicing ---")
5 print("Original 1D Array:", arr_1d)
6 print("Element at index 2:", arr_1d[2])
7 print("Sliced portion [1:4]:", arr_1d[1:4])
8 arr_2d = np.array([[11, 12, 13],
9                    [31, 32, 33]])
10 print("\n--- 2D Array Access ---")
11 print("Original 2D Array:\n", arr_2d)
12 print("Access Specific Row (Row 1):", arr_2d[1])
13 print("Access Specific Column (Col 2):", arr_2d[:, 2])
14 reshaped_arr = arr_1d.reshape(2, 3)
15 print("\n--- Reshaping Array ---")
16 print("Original 1D Array :", arr_1d)
17 print("Reshaped to 2D (2x3):\n", reshaped_arr)
```

rs\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\numpytask2.py
Task 2: NumPy Indexing, Slicing & Reshaping ===

1D Indexing & Slicing ---
Original 1D Array: [10 20 30 40 50 60]
Element at index 2: 30
Sliced portion [1:4]: [20 30 40]

1:19 CRLF UTF-8 4 spaces Types: Built-in ENG

```
task2.py ▾ Version Control ▾ numpytask2 ▾ ▶ 🐍 ⋮ @ 🔍 ⚙️ Trial
```

```
numpytask1.py numpytask2.py ×
```

```
1 import numpy as np
2 print("=== Task 2: NumPy Indexing, Slicing & Reshaping ===")
3 arr_1d = np.array([10, 20, 30, 40, 50, 60])
4 print("\n--- 1D Indexing & Slicing ---")
5 print("Original 1D Array:", arr_1d)
6 print("Element at index 2:", arr_1d[2])
7 print("Sliced portion [1:4]:", arr_1d[1:4])
8 arr_2d = np.array([[11, 12, 13],
9                   [31, 32, 33]])
10 print("\n--- 2D Array Access ---")
11 print("Original 2D Array:\n", arr_2d)
12 print("Access Specific Row (Row 1):", arr_2d[1])
13 print("Access Specific Column (Col 2):", arr_2d[:, 2])
14 reshaped_arr = arr_1d.reshape(2, 3)
15 print("\n--- Reshaping Array ---")
16 print("Original 1D Array :", arr_1d)
17 print("Reshaped to 2D (2x3):\n", reshaped_arr)
```

```
rs\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\numpytask2.py
Task 2: NumPy Indexing, Slicing & Reshaping ===

1D Indexing & Slicing ---
Original 1D Array: [10 20 30 40 50 60]
Element at index 2: 30
Sliced portion [1:4]: [20 30 40]

2D Array Access ---
Original 2D Array:
[[11 12 13]
 [31 32 33]]
Access Specific Row (Row 1): [31 32 33]
Access Specific Column (Col 2): [13 33]

Reshaping Array ---
Original 1D Array : [10 20 30 40 50 60]
Reshaped to 2D (2x3):
[[10 20 30]
 [40 50 60]]
```

1:19 CRLF UTF-8 4 spaces Types: Built-in ENG

Run

numpytask2 x

```
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\numpytask2.py  
=== Task 2: NumPy Indexing, Slicing & Reshaping ===
```

```
--- 1D Indexing & Slicing ---
```

```
Original 1D Array: [10 20 30 40 50 60]
```

```
Element at index 2: 30
```

```
Sliced portion [1:4]: [20 30 40]
```

```
--- 2D Array Access ---
```

```
Original 2D Array:
```

```
[[11 12 13]
```

```
 [31 32 33]]
```

```
Access Specific Row (Row 1): [31 32 33]
```

```
Access Specific Column (Col 2): [13 33]
```

```
--- Reshaping Array ---
```

```
Original 1D Array : [10 20 30 40 50 60]
```

```
Reshaped to 2D (2x3):
```

```
[[10 20 30]
```

```
 [40 50 60]]
```

```
Process finished with exit code 0
```

Desktop > numpytask2.py

1:19 CRLF

Type here to search

```
Run  numpytask2 x
...
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\numpytask2.py
=== Task 2: NumPy Indexing, Slicing & Reshaping ===

--- 1D Indexing & Slicing ---
Original 1D Array: [10 20 30 40 50 60]
Element at index 2: 30
Sliced portion [1:4]: [20 30 40]

--- 2D Array Access ---
Original 2D Array:
[[11 12 13]
 [31 32 33]]
Access Specific Row (Row 1): [31 32 33]
Access Specific Column (Col 2): [13 33]

--- Reshaping Array ---
Original 1D Array : [10 20 30 40 50 60]
Reshaped to 2D (2x3):
[[10 20 30]
 [40 50 60]]

Process finished with exit code 0
```

Desktop > numpytask2.py 1:19 CRL

```
Version Control
numpytask1.py numpytask2.py numpytask3.py x
import numpy as np

print("=== Task 3: NumPy Mathematical & Statistical Operations ===")

arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([2, 4, 5, 10])

print("\n--- 1. Mathematical Operations ---")
print("Array 1:", arr1)
print("Array 2:", arr2)

print("Addition (+):", arr1 + arr2)
print("Subtraction (-):", arr1 - arr2)
print("Multiplication (*):", arr1 * arr2)
print("Division (/):", arr1 / arr2)

dataset = np.array([5, 12, 15, 20, 25, 30, 45])

print("\n--- 2. Statistical Functions ---")
print("Dataset:", dataset)
print("Mean:", np.mean(dataset))
print("Median:", np.median(dataset))
print("Minimum:", np.min(dataset))
print("Maximum:", np.max(dataset))
print("Standard Deviation:", np.round(np.std(dataset), decimals=2))
print("Sum:", np.sum(dataset))

14:42 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.6]
26°C Light rain 11:10 10-08-2026
```

```
Version Control
numpytask3
numpytask1.py numpytask2.py numpytask3.py x
import numpy as np

print("=== Task 3: NumPy Mathematical & Statistical Operations ===")

arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([2, 4, 5, 10])

print("\n--- 1. Mathematical Operations ---")
print("Array 1:", arr1)
print("Array 2:", arr2)

print("Addition (+):      ", arr1 + arr2)
print("Subtraction (-):   ", arr1 - arr2)
print("Multiplication (*): ", arr1 * arr2)
print("Division (/):       ", arr1 / arr2)

dataset = np.array([5, 12, 15, 20, 25, 30, 45])

print("\n--- 2. Statistical Functions ---")
print("Dataset:", dataset)
print("Mean      :", np.mean(dataset))
print("Median    :", np.median(dataset))
print("Minimum   :", np.min(dataset))
print("Maximum   :", np.max(dataset))
print("Standard Deviation :", np.round(np.std(dataset), decimals=2))
print("Sum       :", np.sum(dataset))

14:42 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.6]
```

numpytask2.py Version Control

Project Run numpytask3 numpytask1.py numpytask2.py numpytask3.py

```
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\numpytask3.py
=== Task 3: NumPy Mathematical & Statistical Operations ===

--- 1. Mathematical Operations ---
Array 1: [10 20 30 40]
Array 2: [ 2  4  5 10]
Addition (+): [12 24 35 50]
Subtraction (-): [ 8 16 25 30]
Multiplication (*): [ 20 80 150 400]
Division (/): [5. 5. 6. 4.]

--- 2. Statistical Functions ---
Dataset: [ 5 12 15 20 25 30 45]
Mean : 21.714285714285715
Median : 20.0
Minimum : 5
Maximum : 45
Standard Deviation : 12.21
Sum : 152

Process finished with exit code 0
```

Desktop > numpytask3.py

14:42 CRLF UTF-8 4 spaces

```
PC  NP numpytask2.py  Version Control  numpytask3  Run  numpytask3 x
Project  numpytask1.nv  numpytask2.nv  numpytask3.nv x
Run  numpytask3 x
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\numpytask3.py
=== Task 3: NumPy Mathematical & Statistical Operations ===

--- 1. Mathematical Operations ---
Array 1: [10 20 30 40]
Array 2: [ 2  4  5 10]
Addition (+):      [12 24 35 50]
Subtraction (-):   [ 8 16 25 30]
Multiplication (*): [ 20 80 150 400]
Division (/):      [5. 5. 6. 4.]

--- 2. Statistical Functions ---
Dataset: [ 5 12 15 20 25 30 45]
Mean      : 21.714285714285715
Median    : 20.0
Minimum   : 5
Maximum   : 45
Standard Deviation : 12.21
Sum       : 152

Process finished with exit code 0
|

Desktop > numpytask3.py 14:42 CRLF UTF-8 4 spa
```

```
Version Control v
Desktop
np.Ink
CODE WITH OUTPUT:
I.docx
oles

numpytask1.py numpytask2.py numpytask3.py pandastask4.py x

1 import pandas as pd
2
3
4 print("=== Task 4: Pandas Series & DataFrame ===")
5 print("\n--- 1. Pandas Series ---")
6 data_list = [10, 20, 30, 40, 50]
7 my_series = pd.Series(data_list)
8 print(my_series)
9 print("\n--- 2. Creating DataFrame (Student Info) ---")
10 student_data = {
11     "Name": ["Ayush", "Rahul", "Amit"],
12     "Roll_No": ["0884", "0885", "0886"],
13     "Branch": ["CS", "IT", "CS"]
14 }
15 df = pd.DataFrame(student_data)
16 print("DataFrame:\n", df)
17 print("\nColumn Names:", df.columns.tolist())
18 print("Index Details:", df.index)
19 print("\n--- 3. Adding New Column & Updating ---")
20 df["City"] = ["Bhopal", "Indore", "Gwalior"]
21 print("Updated DataFrame:\n", df)

18:34 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.6]
My dog loves being... 11:21 10-08-2026
```

```
numpytask1.py numpytask2.py numpytask3.py pandastask4.py x
1
2 import pandas as pd
3
4 print("=== Task 4: Pandas Series & DataFrame ===")
5 print("\n--- 1. Pandas Series ---")
6 data_list = [10, 20, 30, 40, 50]
7 my_series = pd.Series(data_list)
8 print(my_series)
9 print("\n--- 2. Creating DataFrame (Student Info) ---")
10 student_data = {
11     "Name": ["Ayush", "Rahul", "Amit"],
12     "Roll_No": ["0884", "0885", "0886"],
13     "Branch": ["CS", "IT", "CS"]
14 }
15 df = pd.DataFrame(student_data)
16 print("DataFrame:\n", df)
17 print("\nColumn Names:", df.columns.tolist())
18 print("Index Details:", df.index)
19 print("\n--- 3. Adding New Column & Updating ---")
20 df["City"] = ["Bhopal", "Indore", "Gwalior"]
21 print("Updated DataFrame:\n", df)
```

I

Types: Built-in venv [3.14.6]

```
numpytask2.py Version Control
Project Run pandastask4
numbtask1.nv numbtask2.nv numbtask3.nv nandastask4.nv
Task 4: Pandas Series & DataFrame
--- 1. Pandas Series ---
0 10
1 20
2 30
3 40
4 50
dtype: int64
--- 2. Creating DataFrame (Student Info) ---
DataFrame:
   Name Roll_No Branch
0  Ayush   0884    CS
1  Rahul   0885    IT
2   Amit   0886    CS
Column Names: ['Name', 'Roll_No', 'Branch']
Index Details: RangeIndex(start=0, stop=3, step=1)
--- 3. Adding New Column & Updating ---
Updated DataFrame:
   Name Roll_No Branch  City
0  Ayush   0884    CS  Bhopal
1  Rahul   0885    IT  Indore
Desktop > pandastask4.py 12:41 CRLF UTF-8 4 spaces Types: Built-in venv [3.14.6]
30°C Partly sunny 11:25 12-08-2026
```

```
numpytask2.py Version Control pandastask4 numvtask1.nv numvtask2.nv numvtask3.nv pandastask4.nv x
Run pandastask4 x
=== Task 4: Pandas Series & DataFrame ===

--- 1. Pandas Series ---
0 10
1 20
2 30
3 40
4 50
dtype: int64

--- 2. Creating DataFrame (Student Info) ---
DataFrame:
   Name Roll_No Branch
0  Ayush   0884    CS
1  Rahul   0885    IT
2  Amit    0886    CS

Column Names: ['Name', 'Roll_No', 'Branch']
Index Details: RangeIndex(start=0, stop=3, step=1)

--- 3. Adding New Column & Updating ---
Updated DataFrame:
   Name Roll_No Branch City
0  Ayush   0884    CS  Bhopal
1  Rahul   0885    IT  Indore
```

Desktop > pandastask4.py 12:41 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.6]

Type here to search 30°C Partly sunny 11:25 10-08-2026

```
Project ▾
Run pandastask4 x
1 20
2 30
3 40
4 50
dtype: int64

--- 2. Creating DataFrame (Student Info) ---
DataFrame:
   Name Roll_No Branch
0  Ayush   0884    CS
1  Rahul   0885    IT
2  Amit    0886    CS

Column Names: ['Name', 'Roll_No', 'Branch']
Index Details: RangeIndex(start=0, stop=3, step=1)

--- 3. Adding New Column & Updating ---
Updated DataFrame:
   Name Roll_No Branch  City
0  Ayush   0884    CS  Bhopal
1  Rahul   0885    IT  Indore
2  Amit    0886    CS  Gwalior

Process finished with exit code 0
```

Desktop > pandastask4.py 12:41 CRLF UTF-8 4 s 30°C Partly sunny

```
Project ▾ numvtask1 nv numvtask2 nv numvtask3 nv pandastask4 nv
Run pandastask4 x
1 20
2 30
3 40
4 50
dtype: int64

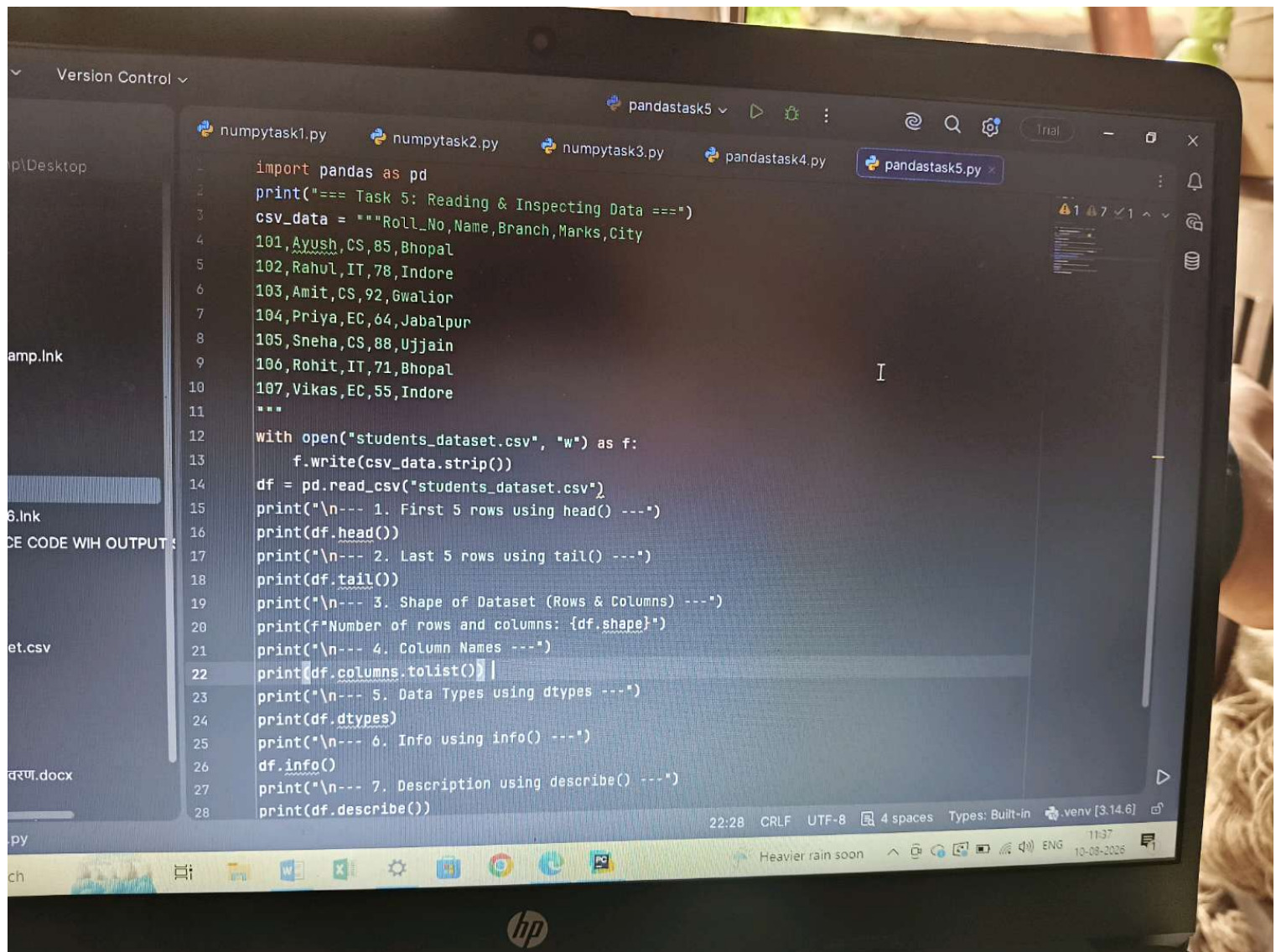
--- 2. Creating DataFrame (Student Info) ---
DataFrame:
   Name Roll_No Branch
0  Ayush   0884    CS
1  Rahul   0885    IT
2   Amit   0886    CS

Column Names: ['Name', 'Roll_No', 'Branch']
Index Details: RangeIndex(start=0, stop=3, step=1)

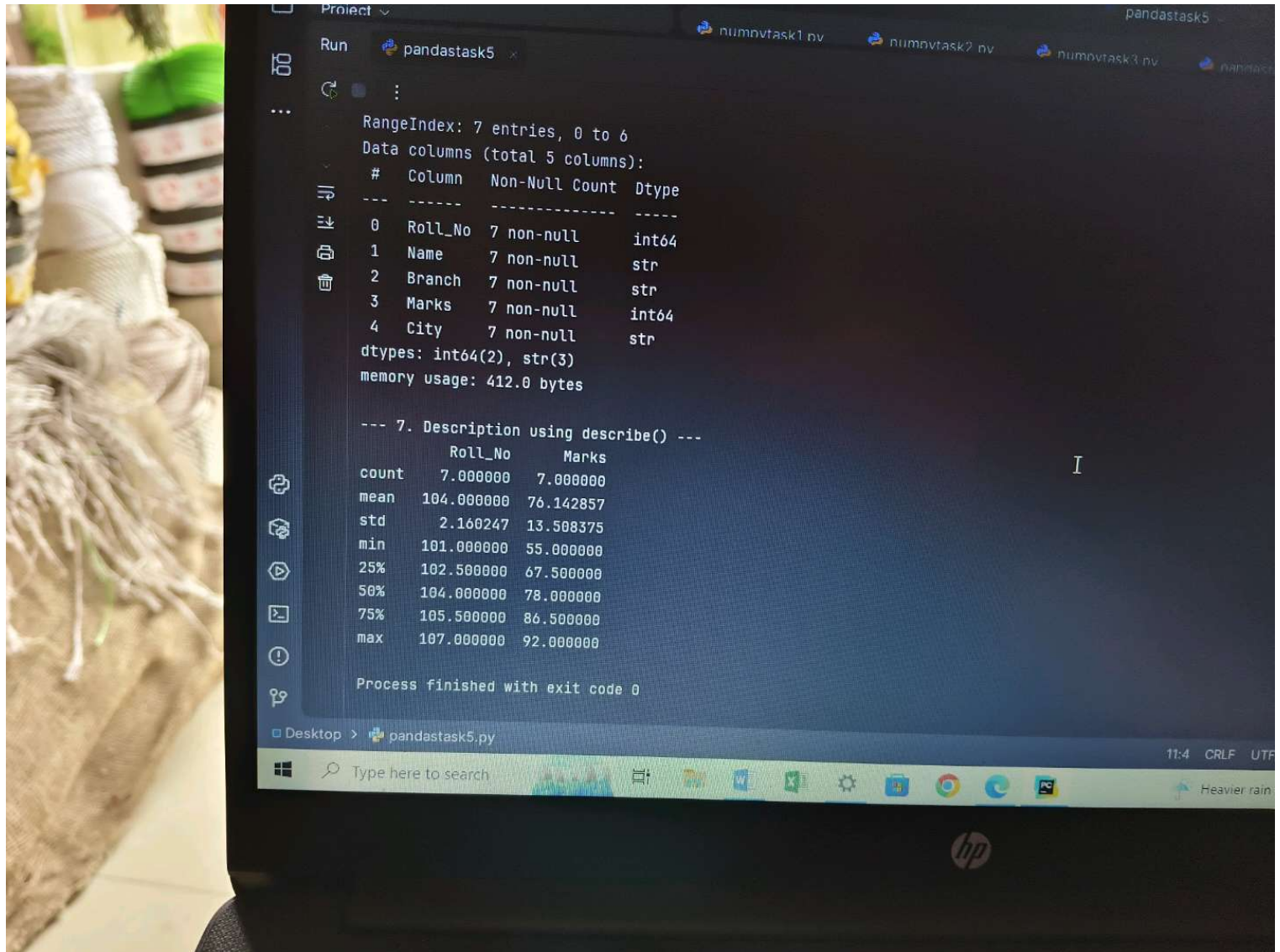
--- 3. Adding New Column & Updating ---
Updated DataFrame:
   Name Roll_No Branch  City
0  Ayush   0884    CS  Bhopal
1  Rahul   0885    IT  Indore
2   Amit   0886    CS  Gwalior

Process finished with exit code 0
```

Desktop > pandastask4.py 12:41 CRLF UTF-8 4 s 30°C Partly sunny



```
Version Control
pandastask5
numpytask1.py numpytask2.py numpytask3.py pandastask4.py pandastask5.py x
import pandas as pd
print("=== Task 5: Reading & Inspecting Data ===")
csv_data = """Roll_No,Name,Branch,Marks,City
101,Ayush,CS,85,Bhopal
102,Rahul,IT,78,Indore
103,Amit,CS,92,Gwalior
104,Priya,EC,64,Jabalpur
105,Sneha,CS,88,Ujjain
106,Rohit,IT,71,Bhopal
107,Vikas,EC,55,Indore
"""
with open("students_dataset.csv", "w") as f:
    f.write(csv_data.strip())
df = pd.read_csv("students_dataset.csv")
print("\n--- 1. First 5 rows using head() ---")
print(df.head())
print("\n--- 2. Last 5 rows using tail() ---")
print(df.tail())
print("\n--- 3. Shape of Dataset (Rows & Columns) ---")
print(f"Number of rows and columns: {df.shape}")
print("\n--- 4. Column Names ---")
print(df.columns.tolist())
print("\n--- 5. Data Types using dtypes ---")
print(df.dtypes)
print("\n--- 6. Info using info() ---")
df.info()
print("\n--- 7. Description using describe() ---")
print(df.describe())
22:28 CRLF UTF-8 4 spaces Types: Built-in venv [3.14.6]
Heavier rain soon 11:37 10-08-2026
```



```
Project
Run pandastask5 x numbvtask1 nv numbvtask2 nv numbvtask3 nv pandastask5

RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
#   Column    Non-Null Count  Dtype
---  -
0   Roll_No   7 non-null      int64
1   Name      7 non-null      str
2   Branch    7 non-null      str
3   Marks     7 non-null      int64
4   City      7 non-null      str
dtypes: int64(2), str(3)
memory usage: 412.0 bytes

--- 7. Description using describe() ---
      Roll_No      Marks
count    7.000000    7.000000
mean   104.000000    76.142857
std     2.160247    13.508375
min    101.000000    55.000000
25%    102.500000    67.500000
50%    104.000000    78.000000
75%    105.500000    86.500000
max    107.000000    92.000000

Process finished with exit code 0
```

Desktop > pandastask5.py

11:4 CRLF UT

Type here to search

Heavier rain

```
umptytask2.py  Version Control  pandastask6  pandastask1.py  numpytask2.py  numpytask3.py  pandastask4.py  pandastask5.py  pandastask6.py

op  Desktop
nv
sktop.ini
cel 2016.lnk
n task.py
t program.py
rth task.py
rosoft Clipchamp.lnk
pytask1.py
pytask2.py
pytask3.py
dastask4.py
dastask5.py
dastask6.py
erPoint 2016.lnk
HON SOURCE CODE WITH OUTPUT
ond task.py
en task.py
ask.py
ents_dataset.csv
8.py
9.py
task.py
d 2016.lnk
साइनमेंट विवरण.docx
ndastask6.py

1 import pandas as pd
2 print("=== Task 6: Selecting, Filtering & Sorting Data ===")
3 student_data = {
4     "Roll_No": [101,102,103,104,105,],
5     "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
6     "Branch": ["CS", "IT", "CS", "EC", "CS"],
7     "Marks": [85, 78, 92, 64, 88]
8 }
9
10 df = pd.DataFrame(student_data)
11 print("Original DataFrame:\n", df)
12 print("\n--- 1. Selecting Specific Columns ---")
13 print(df[["Name", "Marks"]])
14 print("\n--- 2. Selecting Specific Rows (Using iloc) ---")
15 print(df.iloc[0:2])
16 print("\n--- 3. Single Filter (Marks > 80) ---")
17 single_filter = df[df["Marks"] > 80]
18 print(single_filter)
19 print("\n--- 4. Multiple Filters (Branch == CS AND Marks > 85) ---")
20 multiple_filter = df[(df["Branch"] == "CS") & (df["Marks"] > 85)]
21 print(multiple_filter)
22 print("\n--- 5. Sorting in Ascending Order (By Marks) ---")
23 sort_asc = df.sort_values(by="Marks", ascending=True)
24 print(sort_asc)
25 print("\n--- 6. Sorting in Descending Order (By Marks) ---")
26 sort_desc = df.sort_values(by="Marks", ascending=False)
27 print(sort_desc)
```

8:2 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14

re to search 25°C Light rain 12:17 10-08-202

```
numpytask1.py  numpytask2.py  numpytask3.py  pandastask4.py  pandastask5.py  pandastask6.py

1  import pandas as pd
2  print("=== Task 6: Selecting, Filtering & Sorting Data ===")
3  student_data = {
4      "Roll_No": [101,102,103,104,105,],
5      "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
6      "Branch": ["CS", "IT", "CS", "EC", "CS"],
7      "Marks": [85, 78, 92, 64, 88]
8  }
9  df = pd.DataFrame(student_data)
10 print("Original DataFrame:\n", df)
11 print("\n--- 1. Selecting Specific Columns ---")
12 print(df[["Name", "Marks"]])
13 print("\n--- 2. Selecting Specific Rows (Using iloc) ---")
14 print(df.iloc[0:2])
15 print("\n--- 3. Single Filter (Marks > 80) ---")
16 single_filter = df[df["Marks"] > 80]
17 print(single_filter)
18 print("\n--- 4. Multiple Filters (Branch == CS AND Marks > 85) ---")
19 multiple_filter = df[(df["Branch"] == "CS") & (df["Marks"] > 85)]
20 print(multiple_filter)
21 print("\n--- 5. Sorting in Ascending Order (By Marks) ---")
22 sort_asc = df.sort_values(by="Marks", ascending=True)
23 print(sort_asc)
24 print("\n--- 6. Sorting in Descending Order (By Marks) ---")
25 sort_desc = df.sort_values(by="Marks", ascending=False)
26 print(sort_desc)
```

Run

pandastask6

numbvtask1 nv

numbvtask2 nv

pandastask6

numbvtask3 nv

pandastask4 nv

```
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\pandastask6.py
=== Task 6: Selecting, Filtering & Sorting Data ===
```

```
Original DataFrame:
```

	Roll_No	Name	Branch	Marks
0	101	Ayush	CS	85
1	102	Rahul	IT	78
2	103	Amit	CS	92
3	104	Priya	EC	64
4	105	Sneha	CS	88

```
--- 1. Selecting Specific Columns ---
```

	Name	Marks
0	Ayush	85
1	Rahul	78
2	Amit	92
3	Priya	64
4	Sneha	88

```
--- 2. Selecting Specific Rows (Using iloc) ---
```

	Roll_No	Name	Branch	Marks
0	101	Ayush	CS	85
1	102	Rahul	IT	78

```
--- 3. Single Filter (Marks > 80) ---
```

	Roll_No	Name	Branch	Marks
--	---------	------	--------	-------

Desktop > pandastask6.py

8:2 CRLF UTF-8

Type here to search

26°C Light rain

Run

pandastask6

numbtask1 nv

numbtask2 nv

pandastask6

numbtask3 nv

pandastask4 nv

```
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\pandastask6.py
=== Task 6: Selecting, Filtering & Sorting Data ===
```

```
Original DataFrame:
```

	Roll_No	Name	Branch	Marks
0	101	Ayush	CS	85
1	102	Rahul	IT	78
2	103	Amit	CS	92
3	104	Priya	EC	64
4	105	Sneha	CS	88

```
--- 1. Selecting Specific Columns ---
```

	Name	Marks
0	Ayush	85
1	Rahul	78
2	Amit	92
3	Priya	64
4	Sneha	88

```
--- 2. Selecting Specific Rows (Using iloc) ---
```

	Roll_No	Name	Branch	Marks
0	101	Ayush	CS	85
1	102	Rahul	IT	78

```
--- 3. Single Filter (Marks > 80) ---
```

	Roll_No	Name	Branch	Marks
--	---------	------	--------	-------

Desktop > pandastask6.py

8:2 CRLF UTF-8

Type here to search

26°C Light rain

```
PC  NP numpytask2.py Version Control
Project pandastask6
Run pandastask6
4 105 Sneha CS 88

--- 4. Multiple Filters (Branch == CS AND Marks > 85) ---
Roll_No Name Branch Marks
2 103 Amit CS 92
4 105 Sneha CS 88

--- 5. Sorting in Ascending Order (By Marks) ---
Roll_No Name Branch Marks
3 104 Priya EC 64
1 102 Rahul IT 78
0 101 Ayush CS 85
4 105 Sneha CS 88
2 103 Amit CS 92

--- 6. Sorting in Descending Order (By Marks) ---
Roll_No Name Branch Marks
2 103 Amit CS 92
4 105 Sneha CS 88
0 101 Ayush CS 85
1 102 Rahul IT 78
3 104 Priya EC 64

Process finished with exit code 0

Desktop > pandastask6.py 8:2 CRLF UTF-8
Type here to search 26°C Light rain
```

PC NP numpytask2.py Version Control pandastask6

Project numovtask1.py numovtask2.py numovtask3.py pandastask4.py

Run pandastask6

```
4      105  Sneha    CS      88

--- 4. Multiple Filters (Branch == CS AND Marks > 85) ---
Roll_No  Name Branch  Marks
2      103   Amit    CS      92
4      105  Sneha    CS      88

--- 5. Sorting in Ascending Order (By Marks) ---
Roll_No  Name Branch  Marks
3      104  Priya    EC      64
1      102  Rahul    IT      78
0      101  Ayush    CS      85
4      105  Sneha    CS      88
2      103   Amit    CS      92

--- 6. Sorting in Descending Order (By Marks) ---
Roll_No  Name Branch  Marks
2      103   Amit    CS      92
4      105  Sneha    CS      88
0      101  Ayush    CS      85
1      102  Rahul    IT      78
3      104  Priya    EC      64

Process finished with exit code 0
```

Desktop > pandastask6.py 8:2 CRLF UTF-8

Type here to search 26°C Light ra

umpytask2.py

numpytask3.py

pandastask4.py

pandastask6

pandastask5.py

pandastask6.py

pandastask7.py

Trial

```
1 import pandas as pd
2 import numpy as np
3
4 print("=== Task 7: Handling Missing Values ===")
5 student_data = {
6     "Roll_No": [101, 102, 103, 104, 105],
7     "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
8     "Marks": [85, np.nan, 92, np.nan, 76],
9     "City": ["Bhopal", "Indore", None, "Jabalpur", "Bhopal"]
10 }
11 df = pd.DataFrame(student_data)
12 print("\n--- 1. Original Dataset Before Handling ---")
13 print(df)
14 print("\n--- 2. Identifying Missing Values (isnull) ---")
15 print(df.isnull())
16 print("\n--- 3. Count of Missing Values in Each Column ---")
17 print(df.isnull().sum())
18 print("\n--- 4. Dataset After Filling Missing Values (fillna) ---")
19 df_filled = df.copy()
20 df_filled["Marks"] = df_filled["Marks"].fillna(df["Marks"].mean())
21 df_filled["City"] = df_filled["City"].fillna("Unknown")
22 print(df_filled)
23 print("\n--- 5. Dataset After Removing Rows with Missing Values (dropna) ---")
24 df_dropped = df.dropna()
25 print(df_dropped)
```

Project

Run pandastask7

umovtask2 nv

numovtask3 nv

nandastask4 nv

nandastask5 nv

nandastask6 nv

```
C:\Users\hp\Desktop\venv\Scripts\python.exe C:\Users\hp\Desktop\pandastask7.py  
=== Task 7: Handling Missing Values ===
```

```
--- 1. Original Dataset Before Handling ---
```

	Roll_No	Name	Marks	City
0	101	Ayush	85.0	Bhopal
1	102	Rahul	NaN	Indore
2	103	Amit	92.0	NaN
3	104	Priya	NaN	Jabalpur
4	105	Sneha	76.0	Bhopal

```
--- 2. Identifying Missing Values (isnull) ---
```

	Roll_No	Name	Marks	City
0	False	False	False	False
1	False	False	True	False
2	False	False	False	True
3	False	False	True	False
4	False	False	False	False

```
--- 3. Count of Missing Values in Each Column ---
```

```
Roll_No    0  
Name       0  
Marks      2  
City       1  
dtype: int64
```

Desktop > pandastask7.py

16:61 CRLF UTF-8 4 spaces Types

Type here to search

26°C Light rain

```
numpytask2.py v Version Control v
Project v
Run pandastask7 x
umovtask2 nv numovtask3 nv pandastask7 v pandastask4 nv pandastask5 nv pandastask6 nv

3 False False True False
4 False False False False

--- 3. Count of Missing Values in Each Column ---
Roll_No    0
Name        0
Marks       2
City        1
dtype: int64

--- 4. Dataset After Filling Missing Values (fillna) ---
Roll_No  Name  Marks  City
0      101  Ayush  85.000000  Bhopal
1      102  Rahul  84.333333  Indore
2      103  Amit  92.000000  Unknown
3      104  Priya  84.333333  Jabalpur
4      105  Sneha  76.000000  Bhopal

--- 5. Dataset After Removing Rows with Missing Values (dropna) ---
Roll_No  Name  Marks  City
0      101  Ayush  85.0  Bhopal
4      105  Sneha  76.0  Bhopal

Process finished with exit code 0
```

Desktop > pandastask7.py

16:61 CRLF UTF-8 4 spaces Types: Built-in

Type here to search

26°C Light rain

```
numpytask2.py Version Control pandastask7
Project numvtask2.nv numvtask3.nv pandastask4.nv pandastask5.nv pandastask6.nv
Run pandastask7 x
3 False False True False
4 False False False False

--- 3. Count of Missing Values in Each Column ---
Roll_No 0
Name 0
Marks 2
City 1
dtype: int64

--- 4. Dataset After Filling Missing Values (fillna) ---
Roll_No Name Marks City
0 101 Ayush 85.000000 Bhopal
1 102 Rahul 84.333333 Indore
2 103 Amit 92.000000 Unknown
3 104 Priya 84.333333 Jabalpur
4 105 Sneha 76.000000 Bhopal

--- 5. Dataset After Removing Rows with Missing Values (dropna) ---
Roll_No Name Marks City
0 101 Ayush 85.0 Bhopal
4 105 Sneha 76.0 Bhopal

Process finished with exit code 0
```

Desktop > pandastask7.py 16:61 CRLF UTF-8 4 spaces Types: Built- 26°C Light rain

The image shows a Jupyter Notebook interface on a laptop screen. The notebook contains the following Python code:

```

import pandas as pd
import numpy as np

print("=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===")
print("\n--- 1. Merging two DataFrames ---")

df_names = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Name": ["Ayush", "Rahul", "Amit"]
})

df_marks = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Marks": [85, 78, 92]
})

merged_df = pd.merge(df_names, df_marks, on="Roll_No")
print(merged_df)

print("\n--- 2. Concatenating two DataFrames ---")
batch1 = pd.DataFrame({"Name": ["Ayush", "Rahul"], "Branch": ["CS", "IT"]})
batch2 = pd.DataFrame({"Name": ["Amit", "Priya"], "Branch": ["CS", "EC"]})
concated_df = pd.concat([batch1, batch2], ignore_index=True)
print(concated_df)

print("\n--- 3. GroupBy Operations ---")
sales_data = pd.DataFrame({
    "Branch": ["CS", "IT", "CS", "IT", "CS"],
    "Students": ["ayush", "rahul", "amit", "priya", "sneha"],
    "Fees": [5000, 6000, 5200, 5800, 5100]
})

grouped_sum = sales_data.groupby("Branch")["Students"].sum()

```

The taskbar at the bottom of the screen displays various application icons, including a web browser, file explorer, and several terminal or code editor windows. System status information in the bottom right corner indicates a temperature of 27°C, cloudy weather, and the time 12:59 on 10-08-2026.

```
pandas8task
import pandas as pd
import numpy as np

print("=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===")
print("\n--- 1. Merging two DataFrames ---")

df_names = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Name": ["Ayush", "Rahul", "Amit"]
})

df_marks = pd.DataFrame({
    "Roll_No": [101, 102, 103],
    "Marks": [85, 78, 92]
})

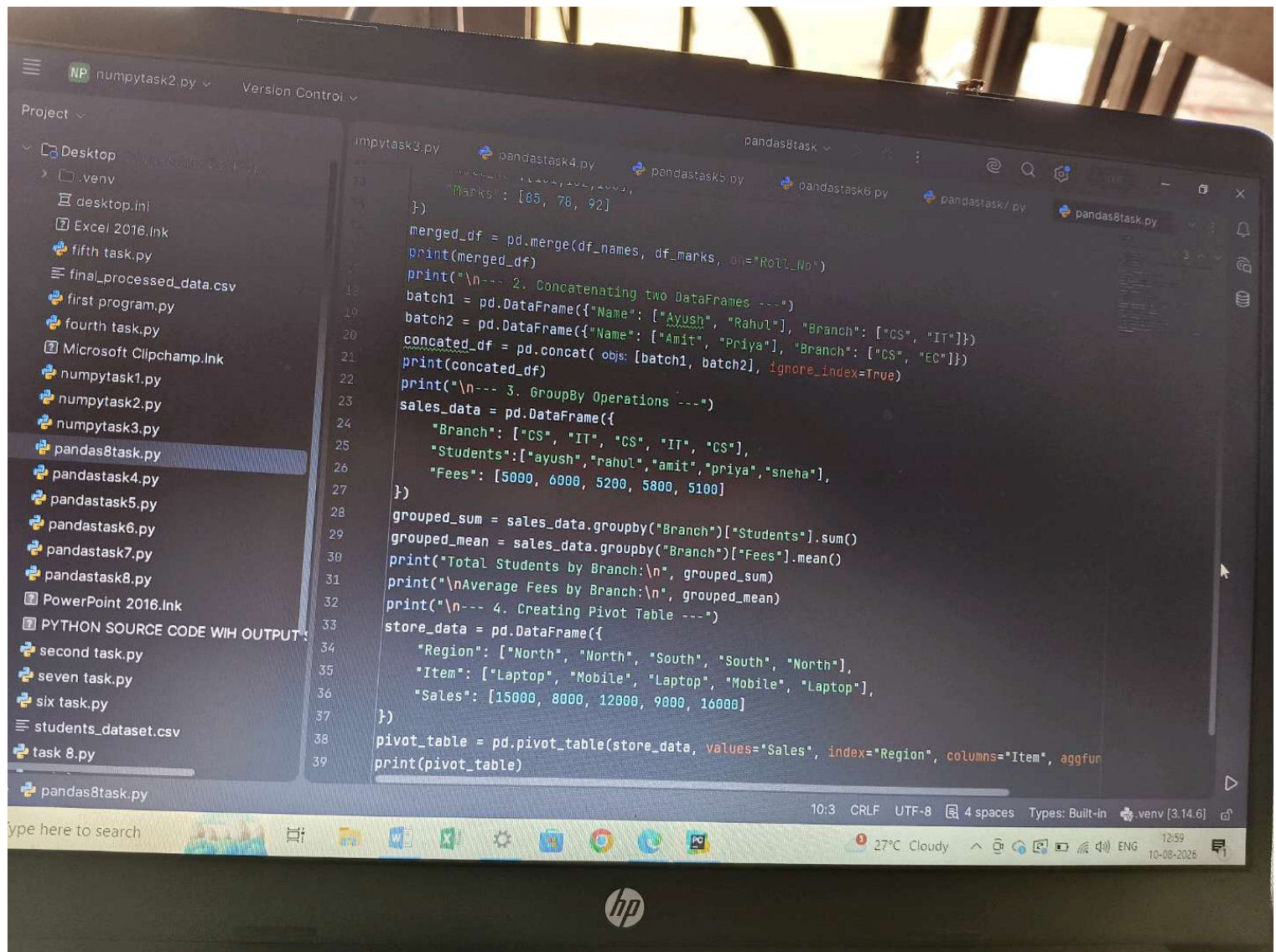
merged_df = pd.merge(df_names, df_marks, on="Roll_No")
print(merged_df)

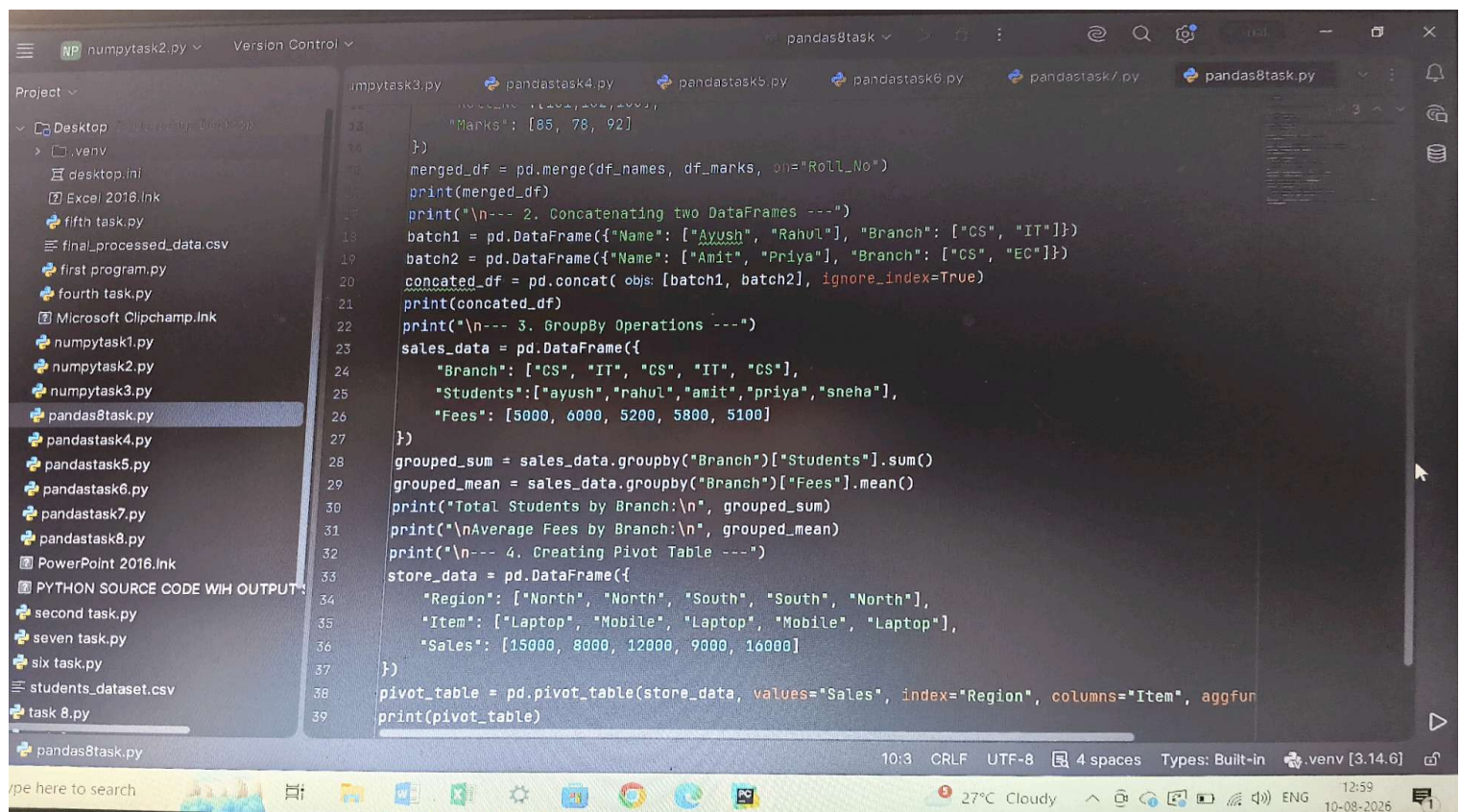
print("\n--- 2. Concatenating two DataFrames ---")
batch1 = pd.DataFrame({"Name": ["Ayush", "Rahul"], "Branch": ["CS", "IT"]})
batch2 = pd.DataFrame({"Name": ["Amit", "Priya"], "Branch": ["CS", "EC"]})
concatated_df = pd.concat([batch1, batch2], ignore_index=True)
print(concatated_df)

print("\n--- 3. GroupBy Operations ---")
sales_data = pd.DataFrame({
    "Branch": ["CS", "IT", "CS", "IT", "CS"],
    "Students": ["ayush", "rahul", "amit", "priya", "sneha"],
    "Fees": [5000, 6000, 5200, 5800, 5100]
})

grouped_sum = sales_data.groupby("Branch")["Students"].sum()
```

10:3 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.6]





```
Project: pandas8task
Run: pandas8task
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\pandas8task.py
=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===

--- 1. Merging two DataFrames ---
Roll_No  Name  Marks
0        101  Ayush   85
1        102  Rahul   78
2        103  Amit    92

--- 2. Concatenating two DataFrames ---
Name Branch
0  Ayush   CS
1  Rahul   IT
2  Amit    CS
3  Priya   EC

--- 3. GroupBy Operations ---
Total Students by Branch:
Branch
CS    ayushamitsneha
IT     rahulpriya
Name: Students, dtype: str

Average Fees by Branch:
Branch
```

```
Project: pandas8task
Run: pandas8task

C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\pandas8task.py
=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===

--- 1. Merging two DataFrames ---
   Roll_No  Name  Marks
0       101  Ayush   85
1       102  Rahul   78
2       103   Amit   92

--- 2. Concatenating two DataFrames ---
   Name Branch
0  Ayush   CS
1  Rahul   IT
2   Amit   CS
3  Priya   EC

--- 3. GroupBy Operations ---
Total Students by Branch:
   Branch
CS  ayushamitsneha
IT   rahulpriya
Name: Students, dtype: str

Average Fees by Branch:
   Branch
```

25:57 CRLF UTF-8 4 spaces Types: Built-in

Project

Run pandas8task

```
1 Rahul IT
2 Amit CS
3 Priya EC
```

--- 3. GroupBy Operations ---

Total Students by Branch:

```
Branch
CS    ayushamitsneha
IT    rahulpriya
Name: Students, dtype: str
```

Average Fees by Branch:

```
Branch
CS    5100.0
IT    5900.0
Name: Fees, dtype: float64
```

--- 4. Creating Pivot Table ---

```
Item    Laptop    Mobile
Region
North    31000    8000
South    12000    9000
```

Process finished with exit code 0

Desktop > pandas8task.py

25:57 CRLF UTF-8 4 spaces Types: B

Type here to search

27°C Cloudy



```
Project
Run pandas8task

1 Rahul IT
2 Amit CS
3 Priya EC

--- 3. GroupBy Operations ---
Total Students by Branch:
Branch
CS ayushamitsneha
IT rahulpriya
Name: Students, dtype: str

Average Fees by Branch:
Branch
CS 5100.0
IT 5900.0
Name: Fees, dtype: float64

--- 4. Creating Pivot Table ---
Item Laptop Mobile
Region
North 31000 8000
South 12000 9000

Process finished with exit code 0
```

25:57 CRLF UTF-8 4 spaces Types: B

Desktop > pandas8task.py 27°C Cloudy

```

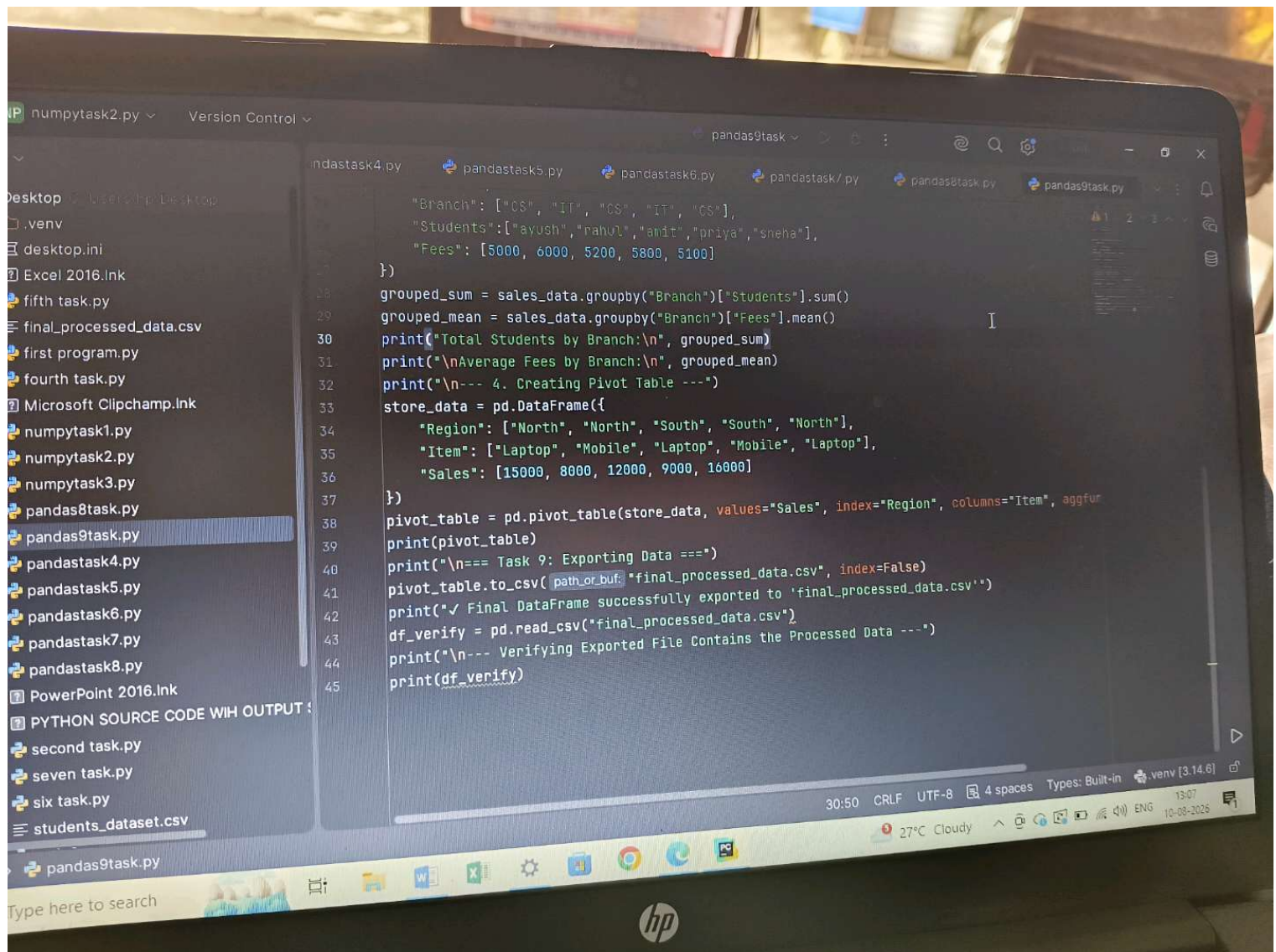
pandas0task.py pandas0task5.py pandas0task6.py pandas0task7.py pandas0task8.py pandas0task9.py
1 import pandas as pd
2 import numpy as np
3
4 print("=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===")
5 print("\n--- 1. Merging two DataFrames ---")
6
7 df_names = pd.DataFrame({
8     "Roll_No": [101, 102, 103],
9     "Name": ["Ayush", "Rahul", "Amit"]
10 })
11 df_marks = pd.DataFrame({
12     "Roll_No": [101, 102, 103],
13     "Marks": [85, 78, 92]
14 })
15 merged_df = pd.merge(df_names, df_marks, on="Roll_No")
16 print(merged_df)
17 print("\n--- 2. Concatenating two DataFrames ---")
18 batch1 = pd.DataFrame({"Name": ["Ayush", "Rahul"], "Branch": ["CS", "IT"]})
19 batch2 = pd.DataFrame({"Name": ["Amit", "Priya"], "Branch": ["CS", "EC"]})
20 concated_df = pd.concat([batch1, batch2], ignore_index=True)
21 print(concated_df)
22 print("\n--- 3. GroupBy Operations ---")
23 sales_data = pd.DataFrame({
24     "Branch": ["CS", "IT", "CS", "IT", "CS"],
25     "Students": ["ayush", "rahul", "amit", "priya", "sneha"],
26     "Fees": [5000, 6000, 5200, 5800, 5100]
27 })
28 grouped_sum = sales_data.groupby("Branch")["Students"].sum()

```

30:50 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.6]

27°C Cloudy 13:07 10-08-2025

```
1 import pandas as pd
2 import numpy as np
3
4 print("=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===")
5 print("\n--- 1. Merging two DataFrames ---")
6
7 df_names = pd.DataFrame({
8     "Roll_No": [101, 102, 103],
9     "Name": ["Ayush", "Rahul", "Amit"]
10 })
11 df_marks = pd.DataFrame({
12     "Roll_No": [101, 102, 103],
13     "Marks": [85, 78, 92]
14 })
15 merged_df = pd.merge(df_names, df_marks, on="Roll_No")
16 print(merged_df)
17 print("\n--- 2. Concatenating two DataFrames ---")
18 batch1 = pd.DataFrame({"Name": ["Ayush", "Rahul"], "Branch": ["CS", "IT"]})
19 batch2 = pd.DataFrame({"Name": ["Amit", "Priya"], "Branch": ["CS", "EC"]})
20 concated_df = pd.concat([batch1, batch2], ignore_index=True)
21 print(concated_df)
22 print("\n--- 3. GroupBy Operations ---")
23 sales_data = pd.DataFrame({
24     "Branch": ["CS", "IT", "CS", "IT", "CS"],
25     "Students": ["ayush", "rahul", "amit", "priya", "sneha"],
26     "Fees": [5000, 6000, 5200, 5800, 5100]
27 })
28 grouped_sum = sales_data.groupby("Branch")["Students"].sum()
```

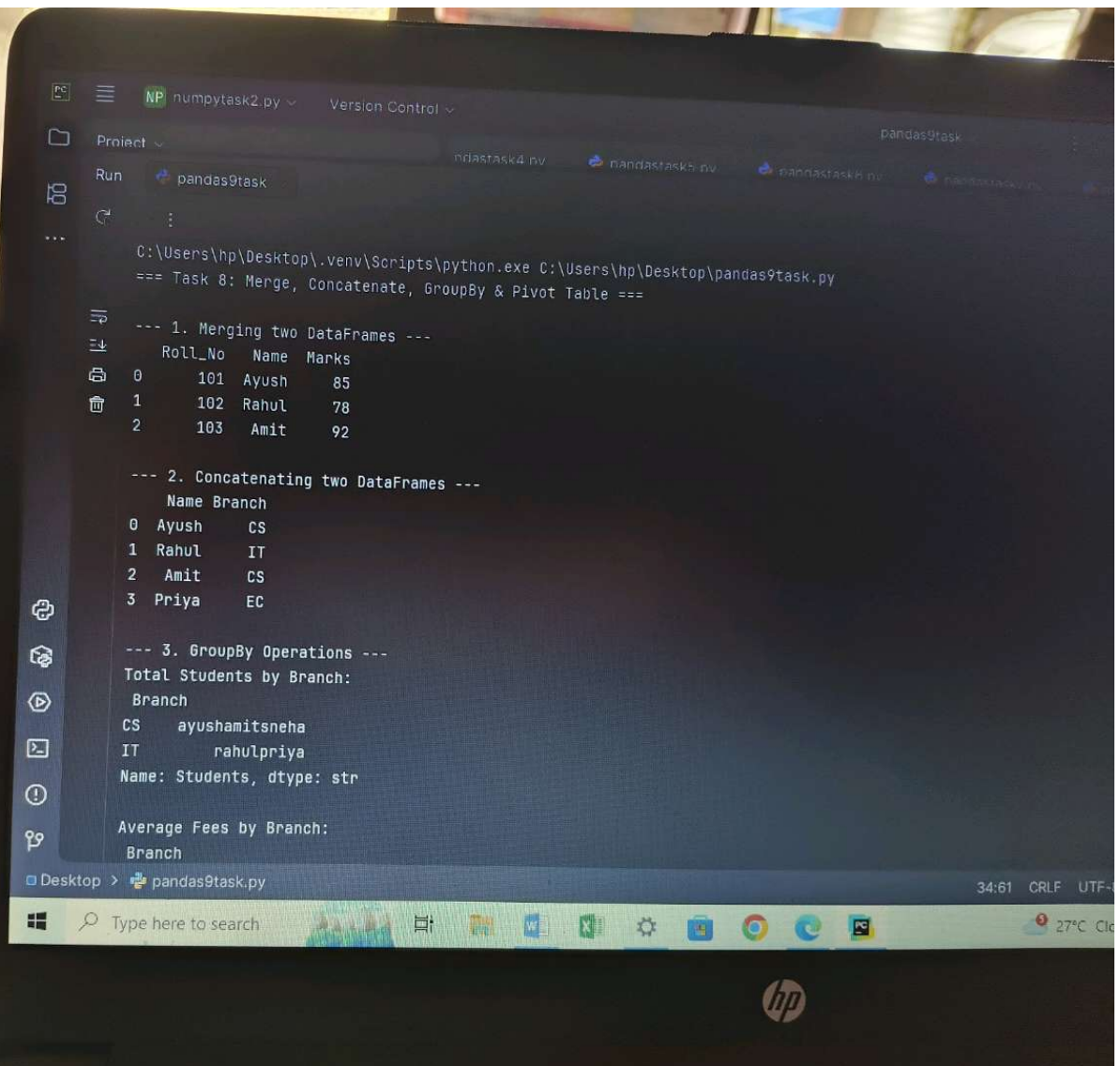


```

    "Branch": ["CS", "IT", "CS", "IT", "CS"],
    "Students": ["ayush", "rahul", "amit", "priya", "sneha"],
    "Fees": [5000, 6000, 5200, 5800, 5100]
})

grouped_sum = sales_data.groupby("Branch")["Students"].sum()
grouped_mean = sales_data.groupby("Branch")["Fees"].mean()
print("Total Students by Branch:\n", grouped_sum)
print("\nAverage Fees by Branch:\n", grouped_mean)
print("\n--- 4. Creating Pivot Table ---")
store_data = pd.DataFrame({
    "Region": ["North", "North", "South", "South", "North"],
    "Item": ["Laptop", "Mobile", "Laptop", "Mobile", "Laptop"],
    "Sales": [15000, 8000, 12000, 9000, 16000]
})
pivot_table = pd.pivot_table(store_data, values="Sales", index="Region", columns="Item", aggfunc="sum")
print(pivot_table)
print("\n== Task 9: Exporting Data ==")
pivot_table.to_csv(path_or_buf="final_processed_data.csv", index=False)
print("✓ Final DataFrame successfully exported to 'final_processed_data.csv'")
df_verify = pd.read_csv("final_processed_data.csv")
print("\n--- Verifying Exported File Contains the Processed Data ---")
print(df_verify)

```



```
NP numpytask2.py Version Control pandas9task

Project ndastask4 nv pandastask5 nv pandastask6 nv pandastask7 nv

Run pandas9task

C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\pandas9task.py
=== Task 8: Merge, Concatenate, GroupBy & Pivot Table ===

--- 1. Merging two DataFrames ---
   Roll_No  Name  Marks
0      101  Ayush   85
1      102  Rahul   78
2      103   Amit   92

--- 2. Concatenating two DataFrames ---
   Name Branch
0  Ayush   CS
1  Rahul   IT
2   Amit   CS
3  Priya   EC

--- 3. GroupBy Operations ---
Total Students by Branch:
   Branch
CS  ayushamitsneha
IT   rahulpriya
Name: Students, dtype: str

Average Fees by Branch:
   Branch
```

```
NP numpytask2.py Version Control
Project pandas9task
Run pandas9task
IT rahulpriya
Name: Students, dtype: str

Average Fees by Branch:
Branch
CS 5100.0
IT 5900.0
Name: Fees, dtype: float64

--- 4. Creating Pivot Table ---
Item Laptop Mobile
Region
North 31000 8000
South 12000 9000

=== Task 9: Exporting Data ===
✓ Final DataFrame successfully exported to 'final_processed_data.csv'

--- Verifying Exported File Contains the Processed Data ---
Laptop Mobile
0 31000 8000
1 12000 9000

Process finished with exit code 0
```

Desktop > pandas9task.py 34:61 CRLF UTF-8 4 spa 27°C Cloudy

```
NP numpytask2.py Version Control pandas9task
Project ndastask4.py pandastask4.py pandastask5.py pandastask6.py pandastask7.py
Run pandas9task
IT rahulpriya
Name: Students, dtype: str

Average Fees by Branch:
Branch
CS 5100.0
IT 5900.0
Name: Fees, dtype: float64

--- 4. Creating Pivot Table ---
Item Laptop Mobile
Region
North 31000 8000
South 12000 9000

=== Task 9: Exporting Data ===
✓ Final DataFrame successfully exported to 'final_processed_data.csv'

--- Verifying Exported File Contains the Processed Data ---
Laptop Mobile
0 31000 8000
1 12000 9000

Process finished with exit code 0

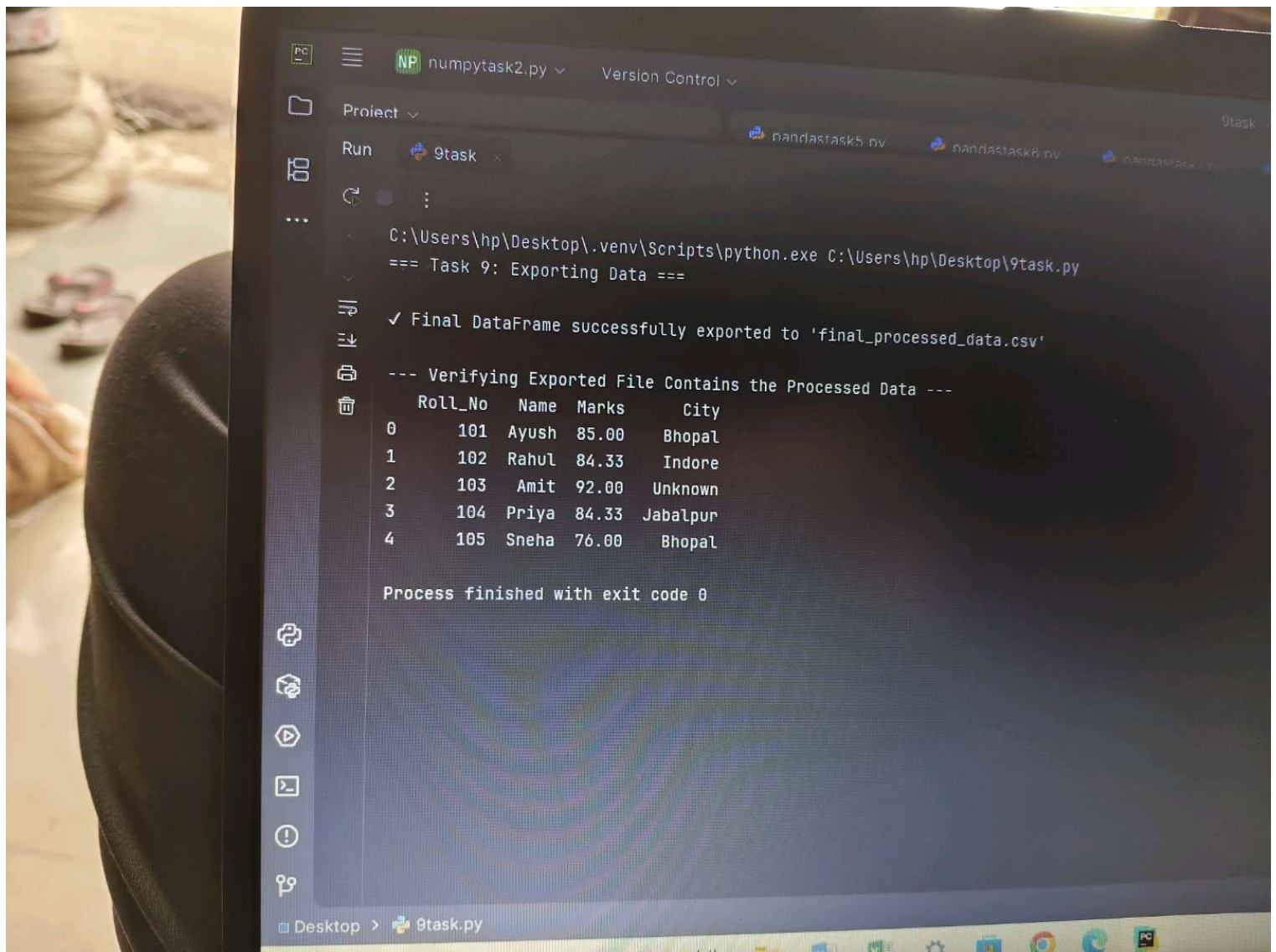
Desktop > pandas9task.py 34:61 CRLF UTF-8 4 spa
Type here to search 27°C Cloudy
```

```
Version Control
pandastask5.py pandastask6.py pandastask7.py pandastask8.py pandastask9.py 9task.py
1 import pandas as pd
2 import numpy as np
3
4 print("=== Task 9: Exporting Data ===")
5
6 cleaned_data = {
7     "Roll_No": [101, 102, 103, 104, 105],
8     "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
9     "Marks": [85.0, 84.33, 92.0, 84.33, 76.0], # साफ़ किया हुआ डेटा
10    "City": ["Bhopal", "Indore", "Unknown", "Jabalpur", "Bhopal"]
11 }
12 df_final = pd.DataFrame(cleaned_data)
13 df_final.to_csv("path_or_buf: final_processed_data.csv", index=False)
14 print("\n✓ Final DataFrame successfully exported to 'final_processed_data.csv'")
15
16 df_verify = pd.read_csv("final_processed_data.csv")
17 print("\n--- Verifying Exported File Contains the Processed Data ---")
18 print(df_verify)
```

8:58 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.6] 13:09 10-08-2026

27°C Cloudy

```
pandastask5.py  pandastask6.py  pandastask7.py  pandas@task.py  pandas@task.py  9task.py
1  import pandas as pd
2  import numpy as np
3
4  print("=== Task 9: Exporting Data ===")
5
6  cleaned_data = {
7      "Roll_No": [101, 102, 103, 104, 105],
8      "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha"],
9      "Marks": [85.0, 84.33, 92.0, 84.33, 76.0], # साफ़ किया हुआ डेटा
10     "City": ["Bhopal", "Indore", "Unknown", "Jabalpur", "Bhopal"]
11 }
12 df_final = pd.DataFrame(cleaned_data)
13 df_final.to_csv(path_or_buf="final_processed_data.csv", index=False)
14 print("\n✓ Final DataFrame successfully exported to 'final_processed_data.csv'")
15
16 df_verify = pd.read_csv("final_processed_data.csv")
17 print("\n--- Verifying Exported File Contains the Processed Data ---")
18 print(df_verify)
```



PCNP numpytask2.pyVersion Control9task

Projectpandastask5.nvpandastask6.nvpandastask7.nv

Run9task

```
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\9task.py
=== Task 9: Exporting Data ===

✓ Final DataFrame successfully exported to 'final_processed_data.csv'

--- Verifying Exported File Contains the Processed Data ---
  Roll_No  Name  Marks  City
0     101  Ayush  85.00  Bhopal
1     102  Rahul  84.33  Indore
2     103   Amit  92.00  Unknown
3     104  Priya  84.33  Jabalpur
4     105  Sneha  76.00  Bhopal

Process finished with exit code 0
```

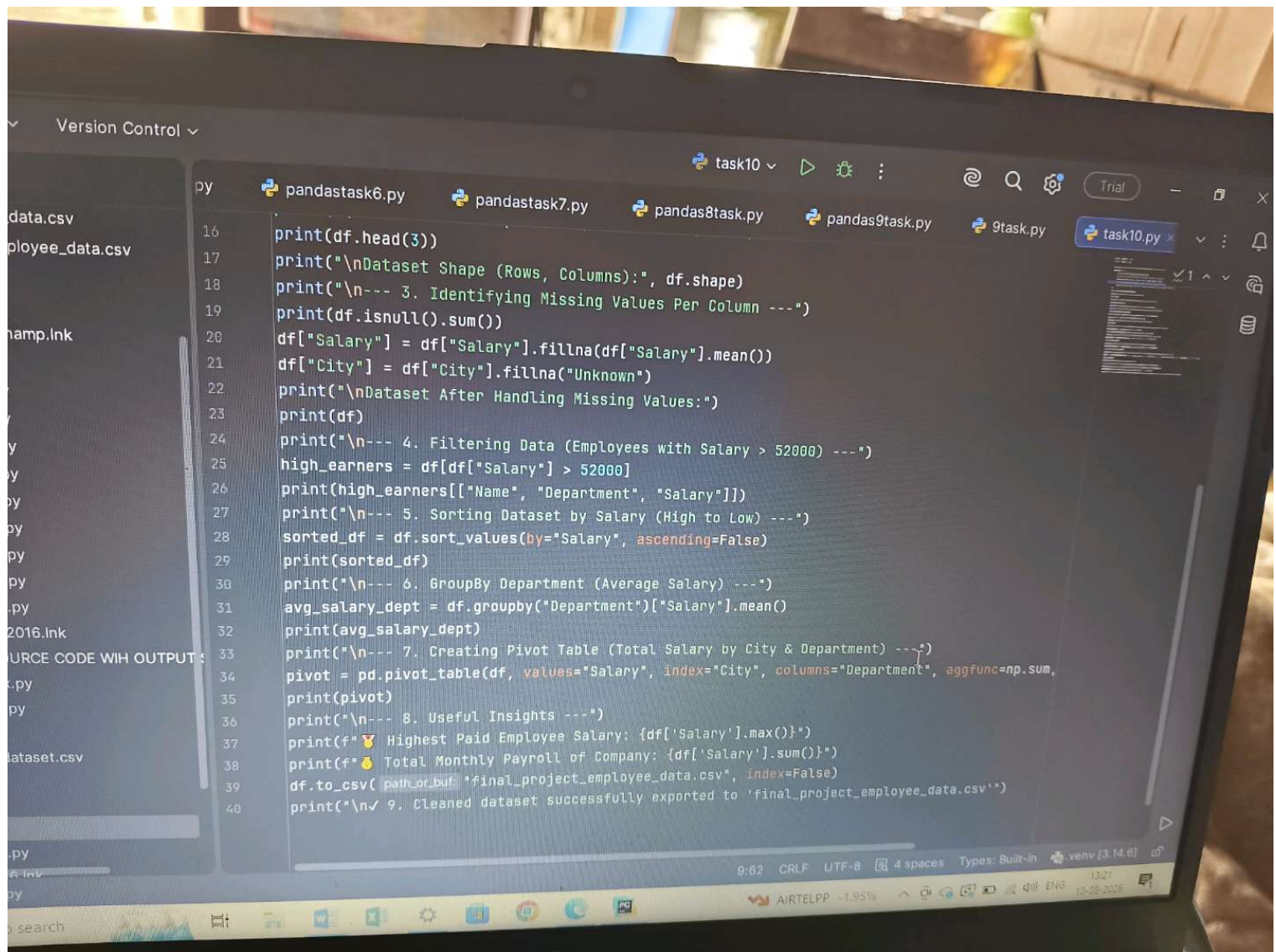
Desktop > 9task.py

```
Version Control v
data.csv
employee_data.csv
mp.lnk
16.lnk
SOURCE CODE WITH OUTPUT:
aset.csv

py
pandastask6.py
pandastask7.py
pandas8task.py
pandas9task.py
9task.py
task10.py

1 import pandas as pd
2 import numpy as np
3
4 print("=== Task 10: Mini Data Analysis Project (Employee Dataset) ===")
5 raw_data = {
6     "Emp_ID": [101, 102, 103, 104, 105, 106],
7     "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha", "Vikas"],
8     "Department": ["IT", "HR", "IT", "HR", "IT", "HR"],
9     "Salary": [60000, np.nan, 75000, 45000, np.nan, 50000],
10    "City": ["Bhopal", "Indore", None, "Jabalpur", "Bhopal", "Indore"]
11 }
12 df = pd.DataFrame(raw_data)
13 print("\n--- 1. Loaded Dataset ---")
14 print(df)
15 print("\n--- 2. Data Inspection (First 3 Rows) ---")
16 print(df.head(3))
17 print("\nDataset Shape (Rows, Columns):", df.shape)
18 print("\n--- 3. Identifying Missing Values Per Column ---")
19 print(df.isnull().sum())
20 df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
21 df["City"] = df["City"].fillna("Unknown")
22 print("\nDataset After Handling Missing Values:")
23 print(df)
24 print("\n--- 4. Filtering Data (Employees with Salary > 52000) ---")
25 high_earners = df[df["Salary"] > 52000]
26 print(high_earners[["Name", "Department", "Salary"]])
27 print("\n--- 5. Sorting Dataset by Salary (High to Low) ---")
28 sorted_df = df.sort_values(by="Salary", ascending=False)
```

```
Version Control
task10
pandastask6.py pandastask7.py pandas8task.py pandas9task.py 9task.py task10.py x
data.csv
oyee_data.csv
mp.lnk
1 import pandas as pd
2 import numpy as np
3
4 print("=== Task 10: Mini Data Analysis Project (Employee Dataset) ===")
5 raw_data = {
6     "Emp_ID": [101, 102, 103, 104, 105, 106],
7     "Name": ["Ayush", "Rahul", "Amit", "Priya", "Sneha", "Vikas"],
8     "Department": ["IT", "HR", "IT", "HR", "IT", "HR"],
9     "Salary": [60000, np.nan, 75000, 45000, np.nan, 50000],
10    "City": ["Bhopal", "Indore", None, "Jabalpur", "Bhopal", "Indore"]
11 }
12 df = pd.DataFrame(raw_data)
13 print("\n--- 1. Loaded Dataset ---")
14 print(df)
15 print("\n--- 2. Data Inspection (First 3 Rows) ---")
16 print(df.head(3))
17 print("\nDataset Shape (Rows, Columns):", df.shape)
18 print("\n--- 3. Identifying Missing Values Per Column ---")
19 print(df.isnull().sum())
20 df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
21 df["City"] = df["City"].fillna("Unknown")
22 print("\nDataset After Handling Missing Values:")
23 print(df)
24 print("\n--- 4. Filtering Data (Employees with Salary > 52000) ---")
25 high_earners = df[df["Salary"] > 52000]
26 print(high_earners[["Name", "Department", "Salary"]])
27 print("\n--- 5. Sorting Dataset by Salary (High to Low) ---")
28 sorted_df = df.sort_values(by="Salary", ascending=False)
9:62 CRLF UTF-8 4 spaces Types: Built-in venv [3.14.6]
```



```
task10 ▾ ▶ ⚙️ ⋮ @ 🔍 ⚙️ Trial - 📄
pandastask6.py pandastask7.py pandas8task.py pandas9task.py 9task.py task10.py x ▾ ⋮
6 print(df.head(3))
7 print("\nDataset Shape (Rows, Columns):", df.shape)
8 print("\n--- 3. Identifying Missing Values Per Column ---")
9 print(df.isnull().sum())
10 df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
11 df["City"] = df["City"].fillna("Unknown")
12 print("\nDataset After Handling Missing Values:")
13 print(df)
14 print("\n--- 4. Filtering Data (Employees with Salary > 52000) ---")
15 high_earners = df[df["Salary"] > 52000]
16 print(high_earners[["Name", "Department", "Salary"]])
17 print("\n--- 5. Sorting Dataset by Salary (High to Low) ---")
18 sorted_df = df.sort_values(by="Salary", ascending=False)
19 print(sorted_df)
20 print("\n--- 6. GroupBy Department (Average Salary) ---")
21 avg_salary_dept = df.groupby("Department")["Salary"].mean()
22 print(avg_salary_dept)
23 print("\n--- 7. Creating Pivot Table (Total Salary by City & Department) ---")
24 pivot = pd.pivot_table(df, values="Salary", index="City", columns="Department", aggfunc=np.sum,
25 print(pivot)
26 print("\n--- 8. Useful Insights ---")
27 print(f"🏆 Highest Paid Employee Salary: {df['Salary'].max()}")
28 print(f"💰 Total Monthly Payroll of Company: {df['Salary'].sum()}")
29 df.to_csv(path_or_buf="final_project_employee_data.csv", index=False)
30 print("\n✓ 9. Cleaned dataset successfully exported to 'final_project_employee_data.csv'")
```

```
PC  NP numpytask2.py  Version Control  task10  task10  nandastask6 nv  nandastask7 nv  nandas8task nv  nandas9task nv

Run task10 x

C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\task10.py
=== Task 10: Mini Data Analysis Project (Employee Dataset) ===

--- 1. Loaded Dataset ---
Emp_ID  Name  Department  Salary  City
0      101  Ayush      IT      60000.0  Bhopal
1      102  Rahul      HR      NaN      Indore
2      103  Amit       IT      75000.0  NaN
3      104  Priya      HR      45000.0  Jabalpur
4      105  Sneha      IT      NaN      Bhopal
5      106  Vikas      HR      50000.0  Indore

--- 2. Data Inspection (First 3 Rows) ---
Emp_ID  Name  Department  Salary  City
0      101  Ayush      IT      60000.0  Bhopal
1      102  Rahul      HR      NaN      Indore
2      103  Amit       IT      75000.0  NaN

Dataset Shape (Rows, Columns): (6, 5)

--- 3. Identifying Missing Values Per Column ---
Emp_ID      0
Name        0
Department  0
Salary      2
```

```
NP numpytask2.py v Version Control v task10 v
Project v nv nandastask6.nv nandastask7.nv nandas8task.nv nandas9task.nv
Run task10 x
C:\Users\hp\Desktop\.venv\Scripts\python.exe C:\Users\hp\Desktop\task10.py
=== Task 10: Mini Data Analysis Project (Employee Dataset) ===

--- 1. Loaded Dataset ---
Emp_ID  Name Department  Salary  City
0      101  Ayush        IT  60000.0  Bhopal
1      102  Rahul          HR    NaN  Indore
2      103  Amit           IT  75000.0   NaN
3      104  Priya          HR  45000.0  Jabalpur
4      105  Sneha          IT    NaN  Bhopal
5      106  Vikas          HR  50000.0  Indore

--- 2. Data Inspection (First 3 Rows) ---
Emp_ID  Name Department  Salary  City
0      101  Ayush        IT  60000.0  Bhopal
1      102  Rahul          HR    NaN  Indore
2      103  Amit           IT  75000.0   NaN

Dataset Shape (Rows, Columns): (6, 5)

--- 3. Identifying Missing Values Per Column ---
Emp_ID      0
Name         0
Department   0
Salary       2
```

```
Project > task10.py Version Control
task10 task6 task7 task8 task9 task10
Run task10
5 166 Vikas HR 50000.0 Indore
3 104 Priya HR 45000.0 Jabalpur

--- 6. GroupBy Department (Average Salary) ---
Department
HR 50833.333333
IT 64166.666667
Name: Salary, dtype: float64

--- 7. Creating Pivot Table (Total Salary by City & Department) ---
Department HR IT
City
Bhopal 0.0 117500.0
Indore 107500.0 0.0
Jabalpur 45000.0 0.0
Unknown 0.0 75000.0

--- 8. Useful Insights ---
🔥 Highest Paid Employee Salary: 75000.0
💰 Total Monthly Payroll of Company: 345000.0

✓ 9. Cleaned dataset successfully exported to 'final_project_employee_data.csv'

Process finished with exit code 0

Desktop > task10.py 27:62 CRLF UTF-8 4 spaces Types: Built-in venv (3.14.6) 13:21
```

Task 1: NumPy Introduction & Arrays

इस टास्क में हमने सबसे पहले `import numpy as np` लिखकर नम्पाई लाइब्रेरी को प्रोग्राम में शामिल किया। इसके बाद `np.array()` फंक्शन की मदद से एक सिंगल लाइन वाला (1D) और एक टेबल जैसा (2D) एरे बनाया। अंत में कंप्यूटर की इन-बिल्ट प्रॉपर्टीज `.shape` (बनावट), `.size` (कुल संख्या) और `.dtype` (डेटा का प्रकार) का उपयोग करके एरे की पूरी जानकारी स्क्रीन पर प्रिंट की।

Task 2: NumPy Indexing, Slicing & Reshaping

इस टास्क में हमने एरे के अंदर से डेटा निकालना सीखा। इंडेक्स 2 लिखकर एक खास नंबर को ढूंढा (Indexing) और `[1:4]` लिखकर एरे के एक निश्चित हिस्से को काटकर अलग किया (Slicing)। इसके बाद `reshape(2, 3)` फंक्शन का उपयोग करके 6 नंबरों की एक सीधी लाइन को 2 रो और 3 कॉलम वाली व्यवस्थित टेबल में बदल दिया।

Task 3: NumPy Mathematical & Statistical Operations

इस टास्क में हमने दो एरे के बीच आमने-सामने वाले नंबरों का जोड़ (+), घटाव (-), गुणा (*) और भाग (/) किया। इसके साथ ही नम्पाई के स्टेटिस्टिक्स शॉर्टकट्स का उपयोग करके पूरे डेटासेट का औसत (mean), बीच की संख्या (median), सबसे छोटा नंबर (min), सबसे बड़ा नंबर (max) और कुल जोड़ (sum) निकाला।

Task 4: Pandas Series & DataFrame

यहाँ से पांडास लाइब्रेरी शुरू हुई। हमने `pd.Series()` की मदद से डेटा का एक सिंगल खड़ी लाइन वाला कॉलम बनाया। इसके बाद `pd.DataFrame()` का उपयोग करके छात्रों का एक पूरा रिकॉर्ड (नाम, रोल नंबर, ब्रांच) टेबल के रूप में तैयार किया और `df["City"]` लिखकर उस बनी-बनाई टेबल में एक नया कॉलम जोड़कर उसे अपडेट किया।

Task 5: Reading & Inspecting Data

इस टास्क में हमने कंप्यूटर में सुरक्षित एक बाहरी CSV फाइल को `pd.read_csv()` की मदद से प्रोग्राम में लोड किया। डेटा की जाँच करने के लिए `head()` से शुरुआत की 5 लाइनें और `tail()` से आखिरी की 5 लाइनें देखीं। इसके साथ ही `.info()` और `.describe()` का उपयोग करके डेटा की पूरी समरी और गणितीय विश्लेषण को चेक किया।

Task 6: Selecting, Filtering & Sorting Data

इस टास्क में हमने पांडास टेबल में से कुछ खास कॉलम (जैसे नाम और मार्क्स) को अलग से चुनना सीखा। इसके बाद एक निश्चित शर्त (जैसे Marks > 80) लगाकर केवल काम के रिकॉर्ड्स को छाँटा (Filtering)। आखिर में `sort_values()` का उपयोग करके पूरी टेबल को मार्क्स के घटते और बढ़ते क्रम में व्यवस्थित किया।

Task 7: Handling Missing Values

इस टास्क में हमने डेटाबेस की खाली या अधूरी जगहों को संभालना सीखा। सबसे पहले `isnull()` फंक्शन से खाली जगहों की पहचान की और `.sum()` से उनकी गिनती की। इसके बाद `fillna()` का उपयोग करके खाली जगहों पर औसत नंबर भरा और `dropna()` की मदद से उन लाइनों को पूरी तरह हटा दिया जिनमें डेटा अधूरा था।

Task 8: Merge, Concatenate, GroupBy & Pivot Table

इस टास्क में हमने दो अलग-अलग टेबल्स को एक कॉमन कॉलम (Roll_No) मिलाकर एक किया (Merge) और दो अलग-अलग बैच के डेटा को एक के नीचे एक जोड़ा (Concatenate)। इसके बाद `groupby()` से ब्रांच के हिसाब से डेटा को ग्रुप करके औसत निकाला और `pivot_table()` की मदद से एक्सेल जैसी एक समरी टेबल तैयार की।

Task 9: Exporting Data

इस टास्क में हमने डेटा को कंप्यूटर में सेव करना सीखा। पिछले टास्क में जो डेटा हमने पूरी तरह साफ़-सुथरा और प्रोसेस किया था, उसे `to_csv()` फंक्शन की मदद से कंप्यूटर के अंदर `final_processed_data.csv` नाम की एक नई फाइल के रूप में सुरक्षित एक्सपोर्ट कर दिया और उसे वापस रीड करके वेरीफाई भी किया।

Task 10: Mini Data Analysis Project

यह इस असाइनमेंट का अंतिम महा-प्रोजेक्ट था, जिसमें हमने ऊपर सीखे गए सभी टूल्स को एक साथ मिलाकर काम में लिया। हमने कंपनी के कर्मचारियों (Employees) का एक अधूरा डेटासेट लोड किया, उसकी पूरी जाँच की, खाली जगहों को भरा, सैलरी के आधार पर डेटा को फ़िल्टर और सॉर्ट किया, डिपार्टमेंट के हिसाब से GroupBy और Pivot Table बनाई, और अंत में इस पूरे साफ़ किए गए प्रोजेक्ट डेटा को एक नई CSV फाइल में सुरक्षित सेव कर दिया।

आभार और इस पूरे असाइनमेंट को सफलतापूर्वक पूरा करने में मैंने अपनी मेहनत के साथ-साथ AI टूल्स और YouTube की भी मदद ली है। शुरुआत के बेसिक टास्क मैंने

अपनी समझ से कोड किए, और बाकी के एडवांस व पेचीदा टास्क को सही तरीके से समझने, उनके लॉजिक्स को क्लियर करने और सिंटैक्स एरर्स (Syntax Errors) को ठीक करने के लिए मैंने AI और YouTube ट्यूटोरियल्स का सहारा लिया है।